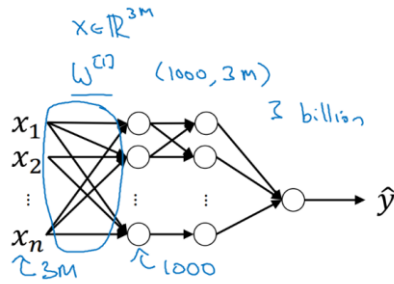


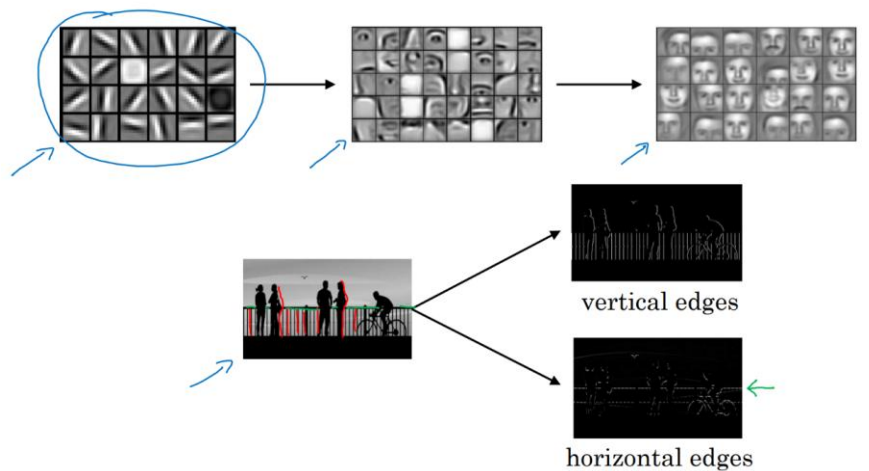
Computer Vision

- Applications: image classification, object detection, neural style transfer
- Deep learning on large images



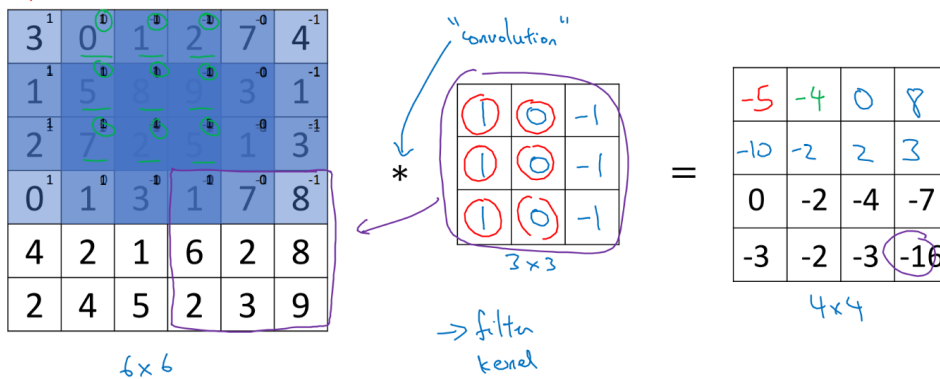
Edge Detection – Convolution Example

- Convolution is a fundamental building block of ML
- Early layers detect edges, middle layers detect objects, later layers detect faces

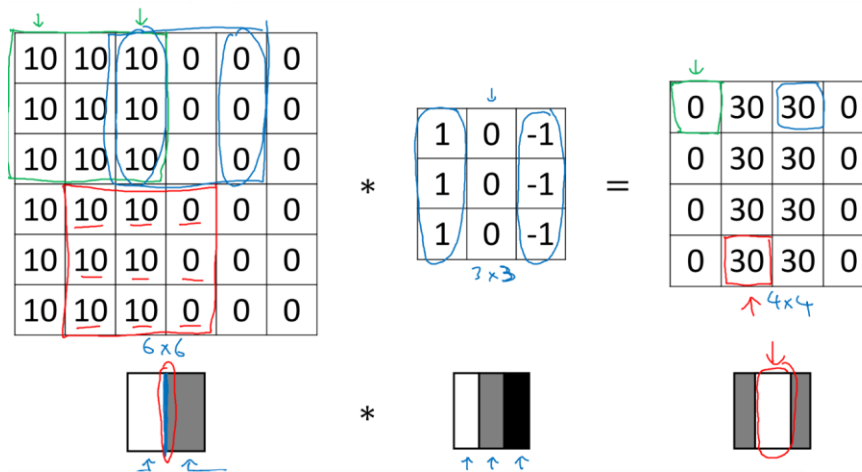


- Vertical edge detection
 - Convolve (*) the input matrix by a **filter/kernel** [1 bright pixels, 0, -1 dark pixels]
 - Steps: paste the filter over the top left corner of the original input, take element-wise product. Adding up these numbers gives the corresponding value of the resulting matrix. Then shift the paste left by one column and repeat for the rest of the row. Then shift the whole block down one row, repeat, etc.
 - In base Python `convolve = conv-forward`
 - Tensorflow: `tf.nn.conv2d`
 - Keras: `Conv2D`

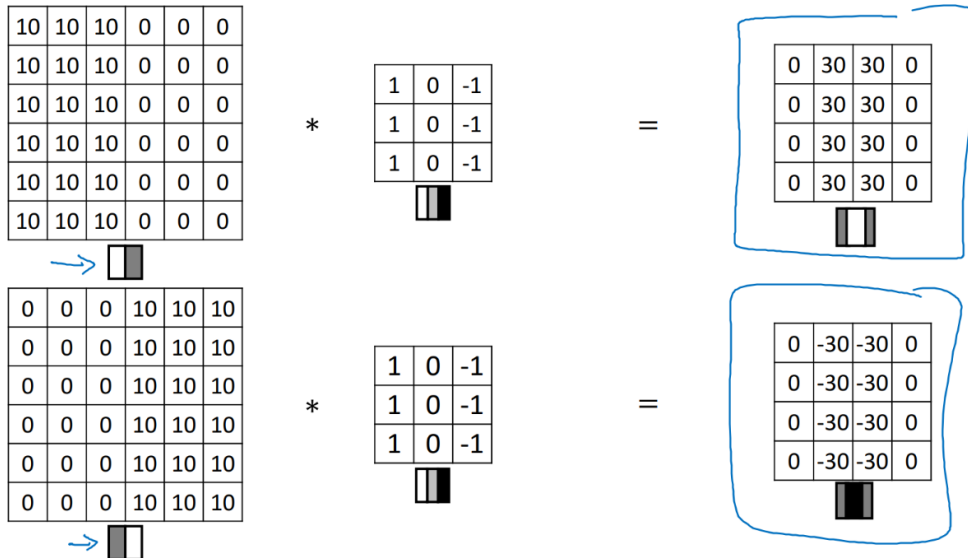
$$\rightarrow 3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times 1 + 8 \times 1 + 2 \times 1 = -5$$



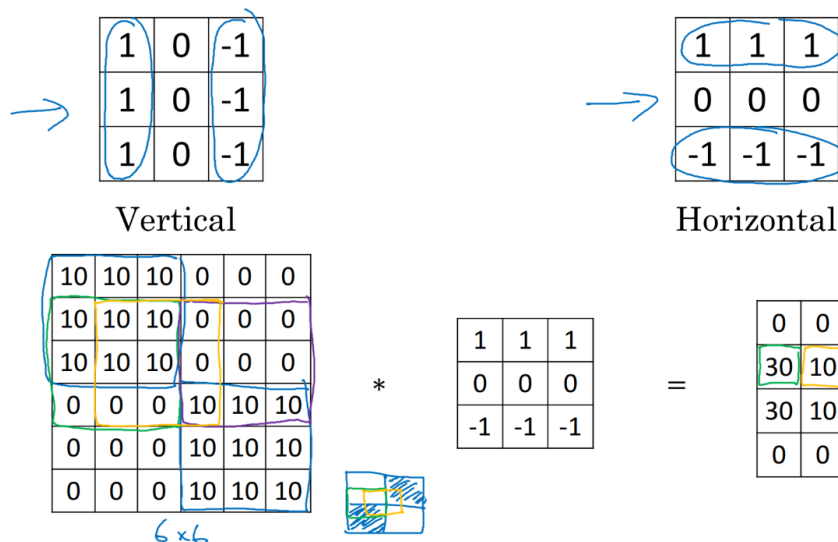
- Example below has a strong vertical edge right down the middle
- Convoluting using the same 3x3 results in 30s down the middle: the vertical edge was detected



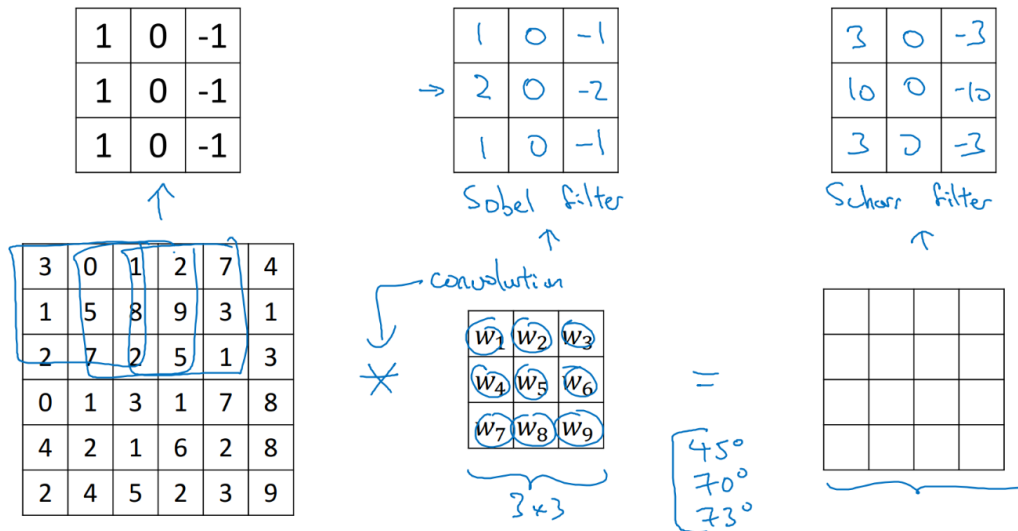
- Dark-to-light vs light-to-dark transitions
 - 10s on left → 30 down middle → light-to-dark transition
 - 10s on right → -30 down middle → dark-to-light transition
 - If you don't care about the direction, you can take the absolute value



- Vertical and horizontal edge detection
 - Vertical: Pixels bright on left and dark on right
 - Horizontal: Pixels bright on top and dark on bottom

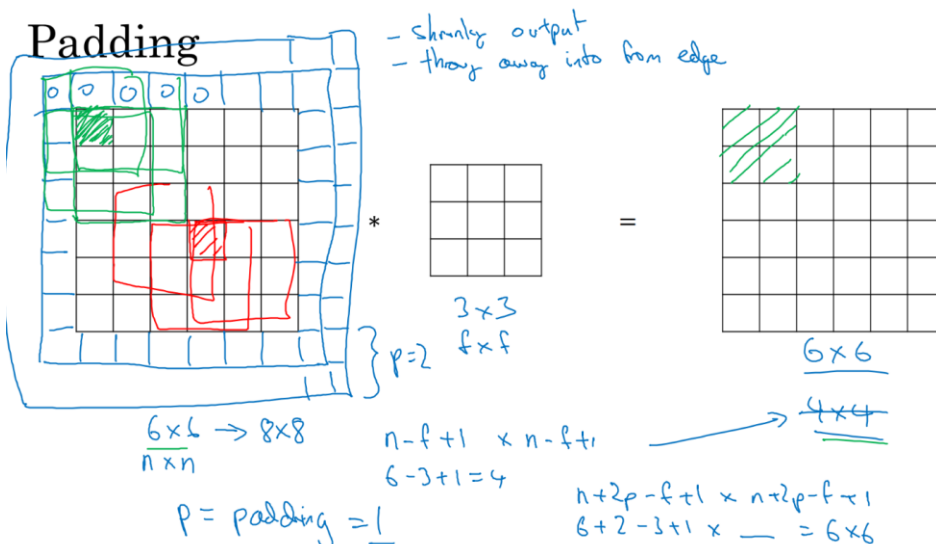


- Learning to detect edges
 - You can use many different filters with different properties (e.g. Sobel, Scharr)
 - Don't have to pick these 9 numbers, you can learn them and treat the 9 numbers of the matrix as parameters that you can learn via backpropagation
 - NN can learn features even more robustly



Padding

- Adding additional border to avoid shrinking the output
- If you have an $n \times n$ image and convolute with an $f \times f$ filter, then the dimension of the output will be $n-f+1 \times n-f+1$
- When padded, the output is $n+2p-f+1 \times n+2p-f+1$



- Two common types of padding for convolutions

- Valid:** no padding

- $p=0$

$$\begin{array}{ccc} n \times n & * & f \times f \rightarrow \frac{n-f+1}{1} \times \frac{n-f+1}{1} \\ 6 \times 6 & * & 3 \times 3 \rightarrow 4 \times 4 \end{array}$$

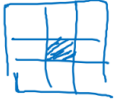
- Same:** pad so that output size is the same as the input size

- $p = (f-1)/2$

- f is usually odd (keeps p even/ adding symmetric, and has a central position to use as a reference for positioning)

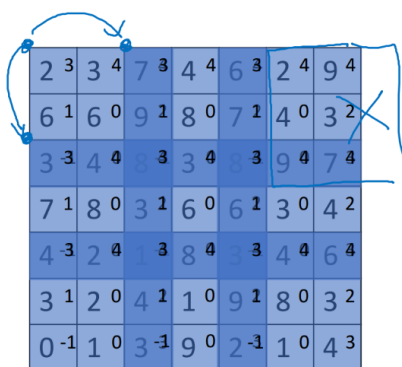
$$\begin{array}{l} n+2p-f+1 \times n+2p-f+1 \\ n+2p-f+1 = n \Rightarrow p = \frac{f-1}{2} \\ 3 \times 3 \quad p = \frac{3-1}{2} = 1 \quad \left| \begin{array}{c} 5 \times 5 \\ f=5 \end{array} \right. \quad p=2 \end{array}$$

f is usually odd



Strided Convolutions

- Moving the box over/down by s steps (instead of default 1 step)
- Summary of convolutions: $n \times n$ image, $f \times f$ filter, padding p , stride s , output size $(n+2p-f)/s + 1 \times (n+2p-f)/s + 1$ (round down if not an integer)
- If any of the box doesn't fit in the matrix, do not include it



$$\begin{array}{|c|c|c|} \hline 3 & 4 & 4 \\ \hline 1 & 0 & 2 \\ \hline -1 & 0 & 3 \\ \hline \end{array} \quad \begin{array}{c} 3 \times 3 \\ \text{stride } s=2 \end{array} = \begin{array}{|c|c|c|} \hline 91 & 100 & 83 \\ \hline 69 & 91 & 127 \\ \hline 44 & 72 & 74 \\ \hline \end{array} \quad \begin{array}{c} 3 \times 3 \end{array}$$

$\lfloor z \rfloor = \text{floor}(z)$

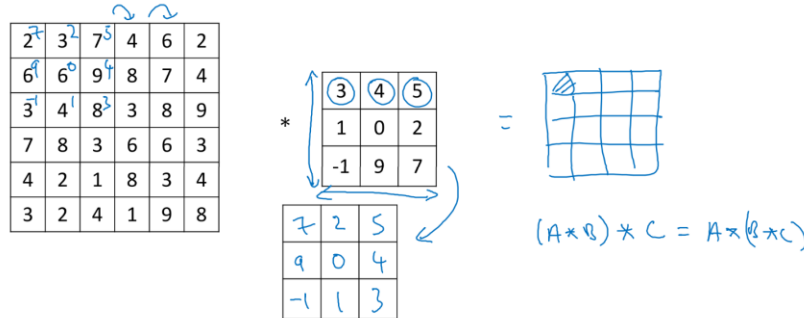
$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

$$\left\lfloor \frac{7+0-3}{2} + 1 \right\rfloor = \frac{4}{2} + 1 = 3$$

$n \times n$ $f \times f$
padding p stride s
 $s=2$

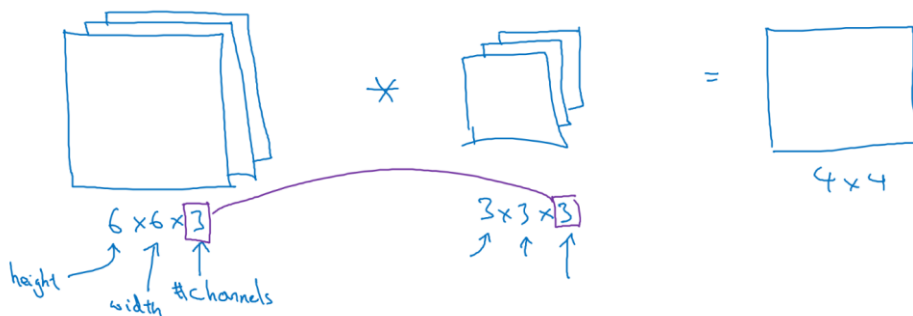
- Technical note on cross-correlation vs convolution
 - Most books transpose the filter matrix before convolving
 - This gives the associativity property $(A * B) * C = A * (B * C)$
 - Not transposing (as done above) is called **cross-correlation**, but by convention in ML they just call it convolution

Convolution in math textbook:

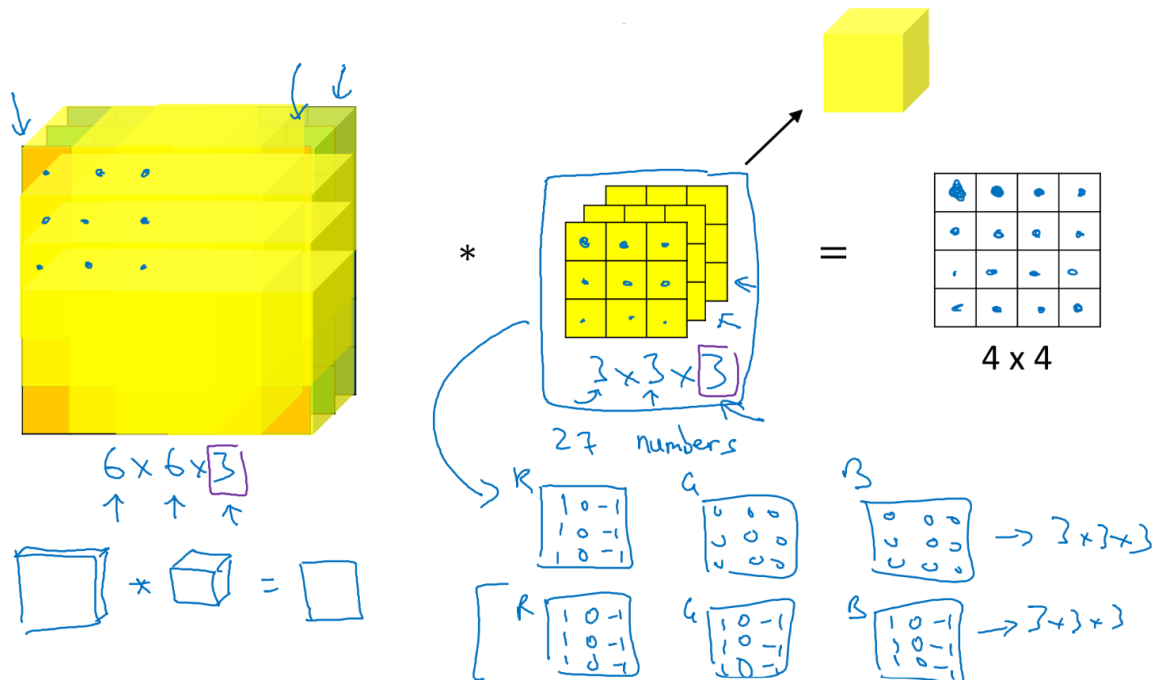


Convolutions Over Volumes

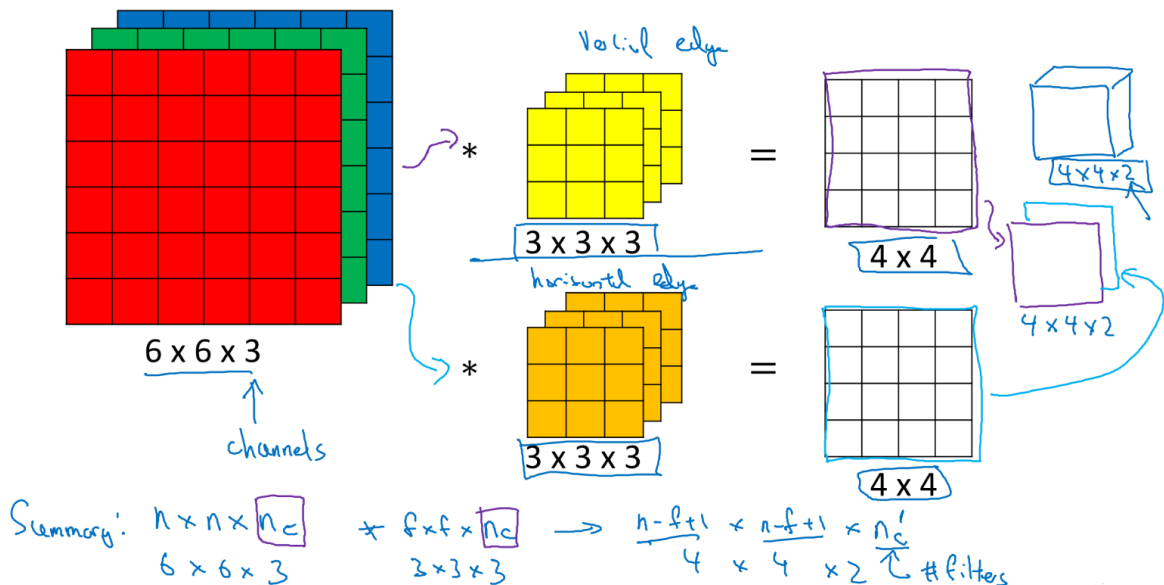
- Convolve with a 3D filter ($3 \times 3 \times 3$)
- Volume dimension/depth (# channels) of input matrix and 3D filter have to be the same
- Output matrix is still flat/volume dimension is 1
- Convolution on RGB images



- 27 filter parameters to multiply by corresponding 27 numbers in the input
- If you want to detect edges in red channel, R filter is $[1 \ 0 \ -1]$, G B filters are $[0 \ 0 \ 0]$
- Edge detector in any color, R C G filters all $[1 \ 0 \ -1]$

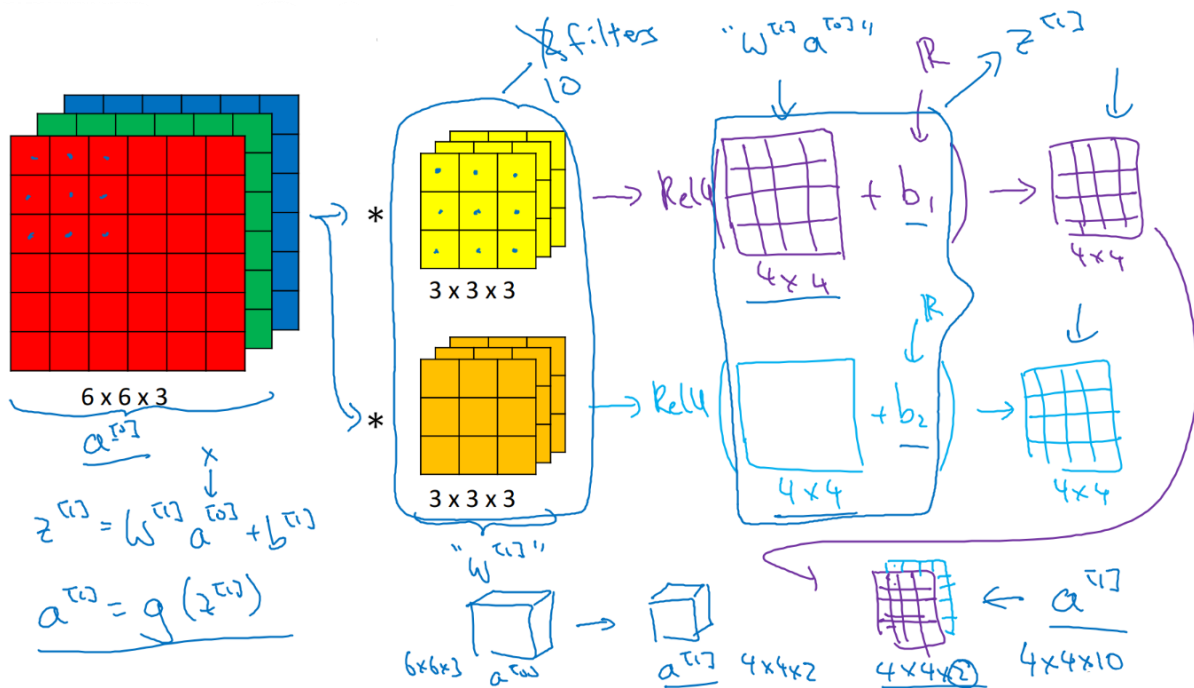


- What if you want to detect vertical edges and horizontal edges (an/or others)?
- Multiple filters
 - Convolve with each filter separately, then stack the filter outputs into a 3D result
 - $n \times n \times n_c * f \times f \times f_c \rightarrow (n-f+1) \times (n-f+1) \times n'_c$ ($n'_c = \# \text{ filters}$; no padding, stride 1)

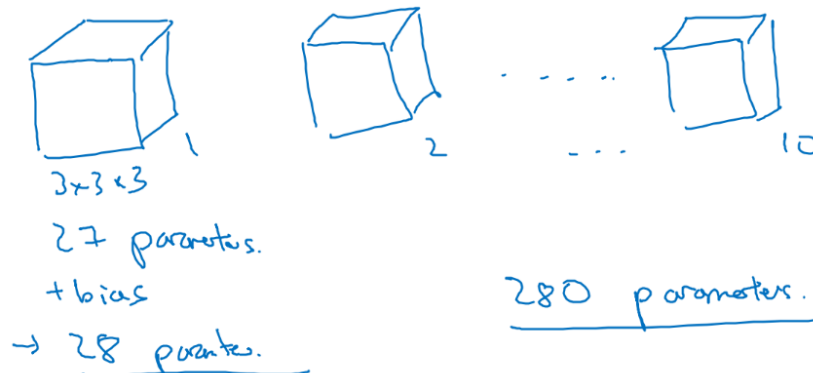


One Layer of a Convolutional Network

- Example
 - Filters play a similar role to $W^{[1]}$, and the input is $a^{[0]}$. Add a bias (role of $z^{[1]}$) and apply non-linearity to each convolution output ($W^{[1]}a^{[0]}$), then stack up ($a^{[1]}$)



- Number of parameters in one layer
 - If you have 10 filters that are $3 \times 3 \times 3$ in one layer of a neural network, how many parameters does that layer have?
 - $(f \times f \times f_c + 1) * n_c$
 - Once you've learned 10 feature detectors that work, you could apply this even to large images, and the number of parameters is still fixed and relatively small



- Summary of notation

If layer l is a convolution layer:

$f^{[l]}$ = filter size
 $p^{[l]}$ = padding
 $s^{[l]}$ = stride
 $n_c^{[l]}$ = number of filters

→ Each filter is: $f^{[l-1]} \times f^{[l-1]} \times n_c^{[l-1]}$
 Activations: $a^{[l-1]} \rightarrow n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$
 Weights: $f^{[l-1]} \times f^{[l-1]} \times n_c^{[l-1]} \times n_c^{[l]}$
 bias: $n_c^{[l]} - (1, 1, 1, n_c^{[l]})$ ← #filters in layer l .

Input: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$
 Output: $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

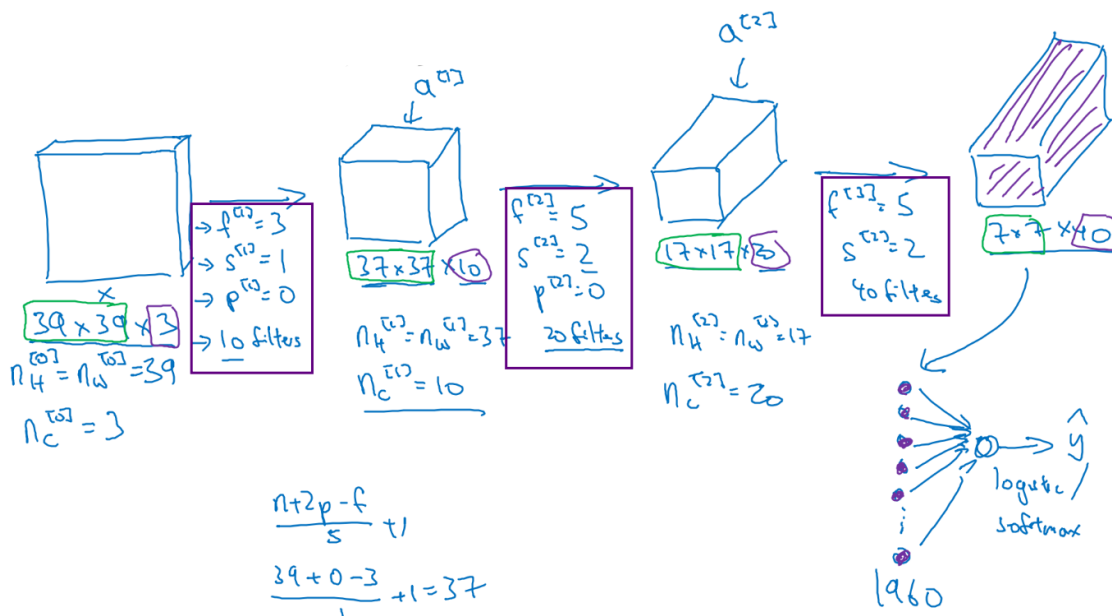
$$n_{HW}^{[l]} = \left\lfloor \frac{n_H^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

$$A^{[l]} \rightarrow m \times n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$$

$$n_c^{[l]} - (1, 1, 1, n_c^{[l]})$$

Simple ConvNet Example

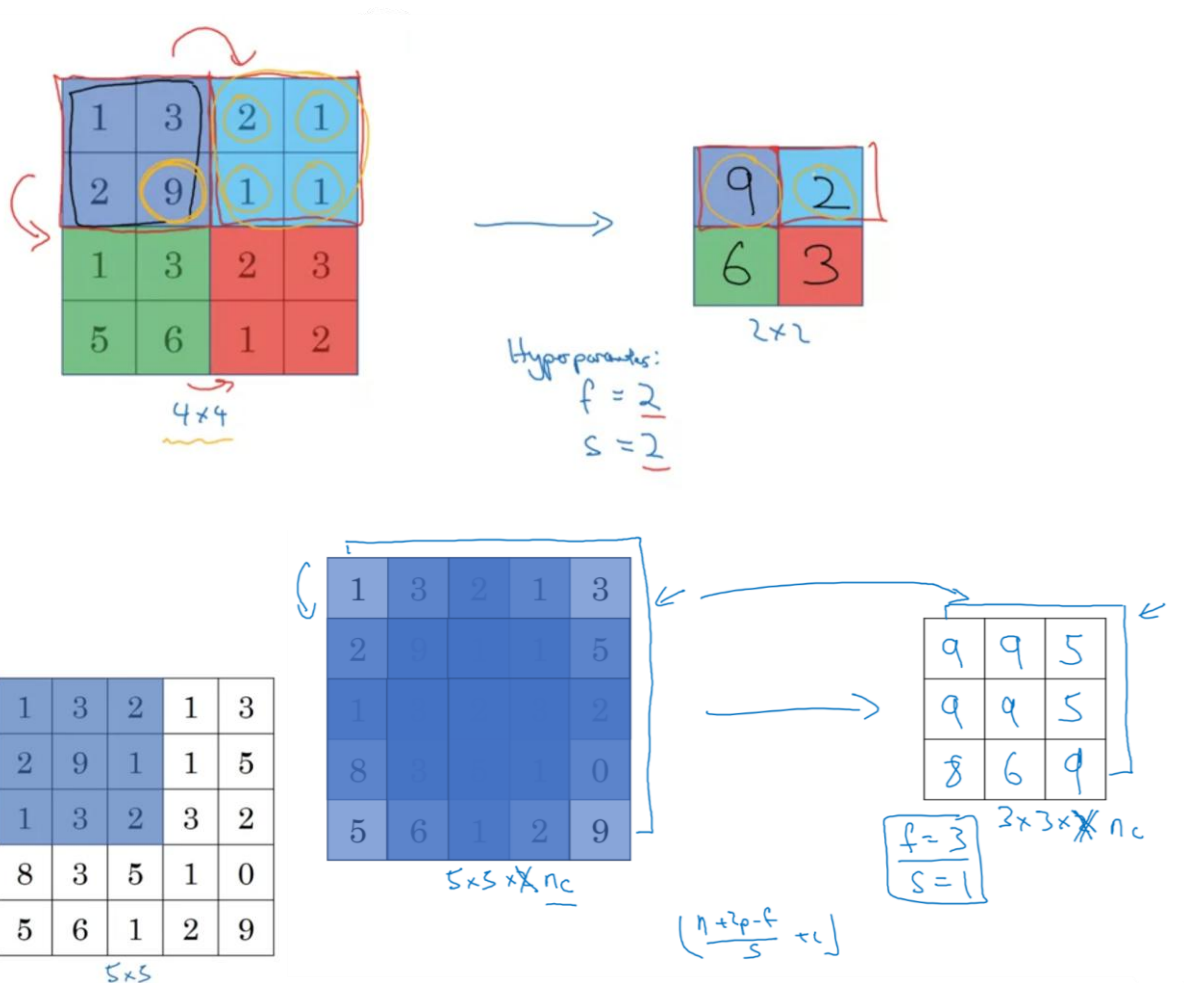
- $p=0 \rightarrow$ valid convolution
- Flatten out final output stack into a vector and feed into logistic or softmax unit to make prediction for final output
- Typically you start with larger images, height and width will stay the same for a while then gradually trend down as you go deeper in the NN, whereas the number of channels will generally increase



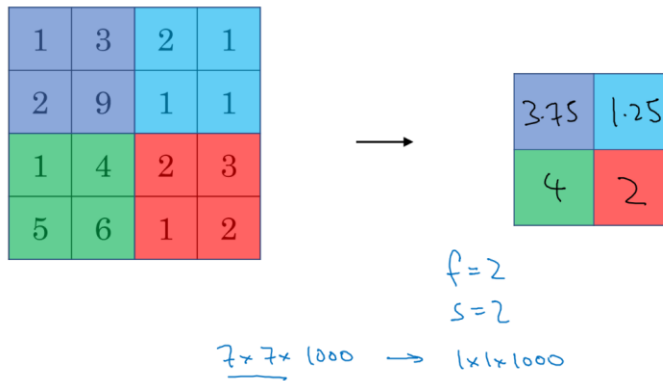
- Types of layer in a convolutional network: Convolution, Pooling, Fully connected (FC)

Pooling Layers

- Reduce the size of the representation to speed up computation as well as make some of the features it detects a bit more robust
- Max pooling
 - Each cell of the output = max from the corresponding shaded region of the input
 - If features are detected anywhere in the quadrant, it remains preserved in the output of max pooling
 - No parameters to learn, no gradient descent, just fixed computation with f and s
 - If 3D input, perform computation on each channel independently



- Average pooling
 - Isn't used very often
 - Very deep in a NN, you might use average pooling to collapse your representation
 - Each cell of the output = average of each corresponding shaded region

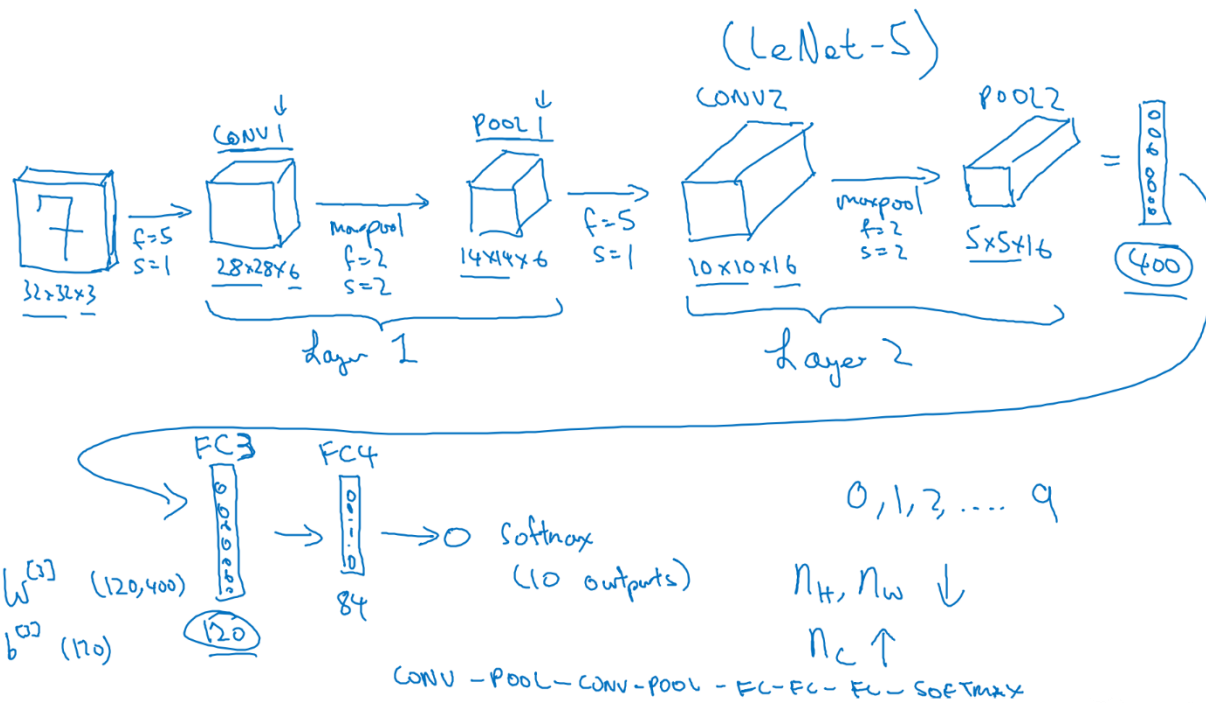


- Summary of pooling
 - Hyperparameters:
 - f = filter size (commonly $f=2$)
 - s = stride (commonly $s=2$)
 - Max or average pooling
 - Can add padding (very rarely used)
 - No parameters to learn!

$$\begin{aligned}
 & n_H \times n_W \times n_C \\
 & \downarrow \\
 & \left\lfloor \frac{n_H - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W - f}{s} + 1 \right\rfloor \\
 & \times n_C
 \end{aligned}$$

ConvNet Example with Gully Connected Layers

- Example inspired by LeNet5 created by Yann LeCun
- When counting layers, people often just count the number of layers that have weights/parameters, so pooling layers can be grouped in (e.g. Conv 1 and Pool 1)
- Image dimensions shrink over time, # channels increases, flatten output into vector of and feed into a fully connected layer (each of the 400 units from the flattened vector is connected to each of the 120 units of the FC layer), feed to another FC layer, then feed into softmax output layer
- Common to have a few Conv $\rightarrow$ Pool and at the end a few FC $\rightarrow$ FC and output softmax

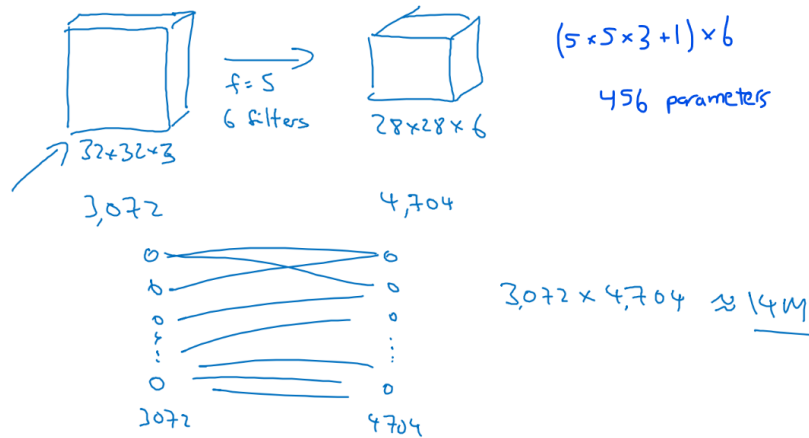


- Pool layers have no parameters, conv layers have few parameters, most parameters are in FC layers, activation size generally decreases throughout

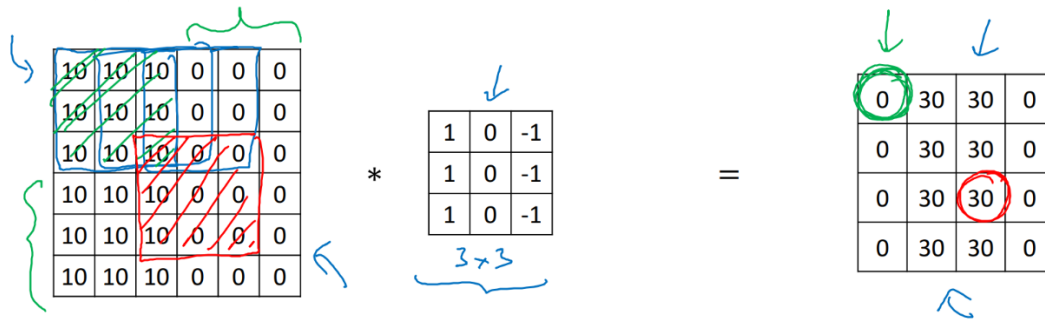
	Activation shape	Activation Size	# parameters
Input:	(32,32,3)	3,072 $a^{(0)}$	0
CONV1 ($f=5, s=1$)	(28,28,6)	4,704	456 ←
POOL1	(14,14,6)	1,176	0 ←
CONV2 ($f=5, s=1$)	(10,10,16)	1,600	2,416 ←
POOL2	(5,5,16)	400	0 ←
FC3	(120,1)	120	48,120
FC4	(84,1)	84	10,164
Softmax	(10,1)	10	850

Why Convolutions?

- Vastly reduces the number of parameters to train

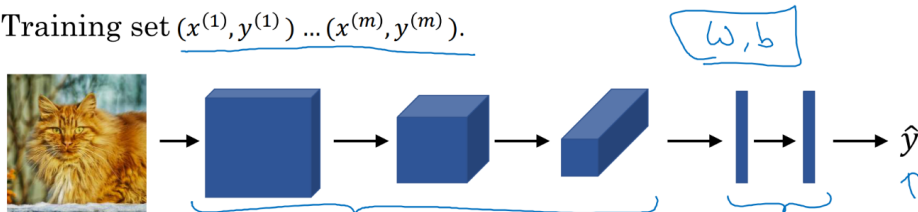


- Advantages (how ConvNets achieve so few parameters)
 - Parameter sharing:** a feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image
 - Sparsity of connections:** in each layer, each output value depends only on a small number of inputs
 - Also, good at capturing **translation invariance:** robust to shifting the image



- Putting it together

Training set $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$.



$$\text{Cost } J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Use gradient descent to optimize parameters to reduce J

What do you think applying this filter to a grayscale image will do?

$$\begin{bmatrix} 0 & 1 & -1 & 0 \\ 1 & 3 & -3 & -1 \\ 1 & 3 & -3 & -1 \\ 0 & 1 & -1 & 0 \end{bmatrix}$$

Detect horizontal edges

Detect image contrast

Detect vertical edges

Detect 45 degree edges

Suppose your input is a 300 by 300 color (RGB) image, and you are not using a convolutional network. If the first hidden layer has 100 neurons, each one fully connected to the input, how many parameters does this hidden layer have (including the bias parameters)?

27,000,100

9,000,100

27,000,001

9,000,001

$$(300 \times 300 \times 3 + 1) \times 100 = (270,000 + 1) \times 100 = 27,000,100$$

Suppose your input is a 256 by 256 grayscale image, and you use a convolutional layer with 128 filters that are each 3×3 . How many parameters does this hidden layer have (including the bias parameters)?

3584

1280

1152

75497600

$$(3 \times 3 \times 1 + 1) \times 128 = (9 + 1) \times 128 = 1280$$

You have an input volume that is $63 \times 63 \times 16$, and convolve it with 32 filters that are each 7×7 , using a stride of 2 and no padding. What is the output volume?

$29 \times 29 \times 32$

$16 \times 16 \times 32$

$29 \times 29 \times 16$

$16 \times 16 \times 16$

$$\text{Output size: } ((n - f + 2p) / s) + 1 = [(63 - 7 + 2(0)) / 2] + 1 = 29 \text{ \# Filters: } 32$$

You have an input volume that is 15×15×8, and pad it using “pad=2”. What is the dimension of the resulting volume (after padding)?

19×19×12 **19×19×8** 17×17×10 17×17×8

$$(n + 2p) \times (n + 2p) \times c = (15 + 2(2)) \times (15 + 2(2)) \times 8 = 19 \times 19 \times 8$$

You have an input volume that is 63×63×16, and convolve it with 32 filters that are each 7×7, and stride of 1. You want to use a “same” convolution. What is the padding?

1 2 7 **3**

$$p = (f - 1) / 2p = (7 - 1) / 2 = 3$$

You have an input volume that is 66×66×21, and apply max pooling with a stride of 3 and a filter size of 3. What is the output volume?

22 × 22 × 21

21 × 21 × 21

22 × 22 × 7

66 × 66 × 7

$$\text{Output size: } ((n - f) / s) + 1 = ((66 - 3) / 3) + 1 = 22 \text{ Filters: } 21$$

Which of the following are hyperparameters of the pooling layers? (Choose all that apply)

Average weights.

Filter size.

Number of filters.

Whether it is max or average.

In lecture we talked about “parameter sharing” as a benefit of using convolutional networks. Which of the following statements about parameter sharing in ConvNets are true? (Check all that apply)

It allows parameters learned for one task to be shared even for a different task (transfer learning).

It allows gradient descent to set many of the parameters to zero, thus making the connections sparse.

It allows a feature detector to be used in multiple locations throughout the whole input image/input volume.

It reduces the total number of parameters, thus reducing overfitting.

The sparsity of connections and weight sharing are mechanisms that allow us to use fewer parameters in a convolutional layer making it possible to train a network with smaller training sets. True/False?

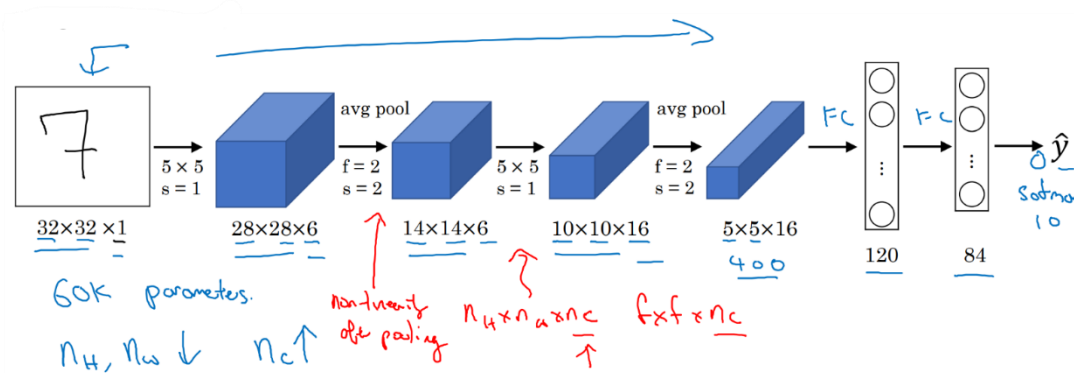
False

True

Deep Convolutional Models: Classic Networks

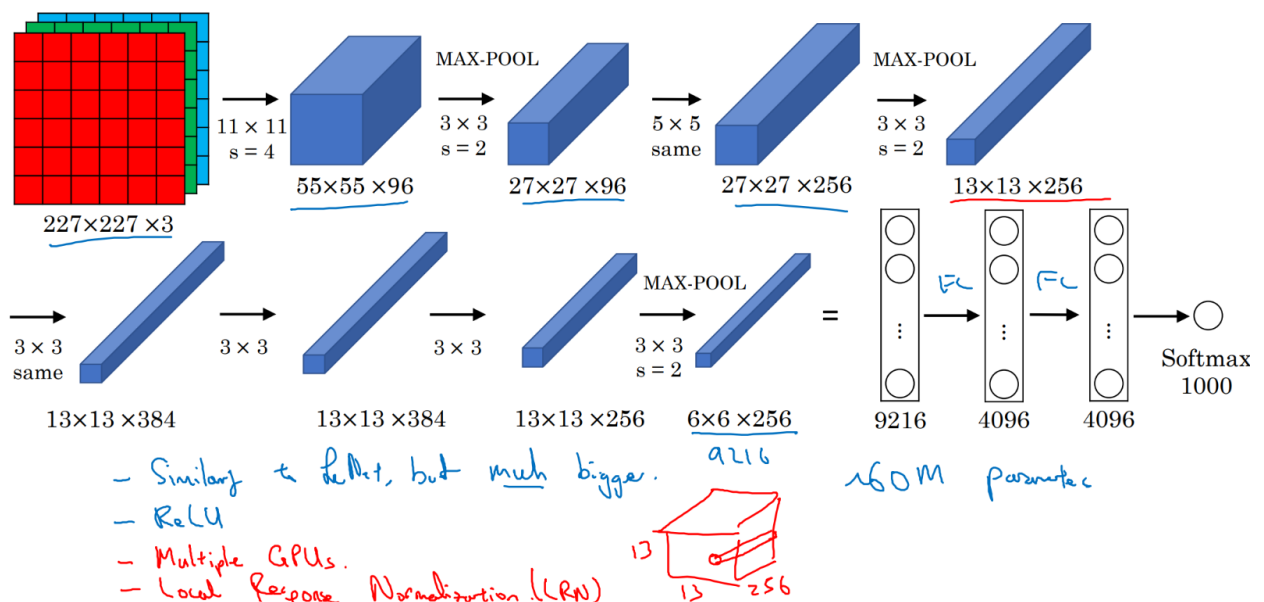
• LeNet-5

- Recognizing hand-written digits from grayscale images
- Final softmax output (used an archaic equivalent at the time) for digit from 0-9
- Avg pooling no padding; 60,000 parameters; high and width decrease but number of channels increase deeper into NN; conv pool conv pool fc fc output
- Non-linearity after pooling layer; had complicated filter application that isn't necessary today



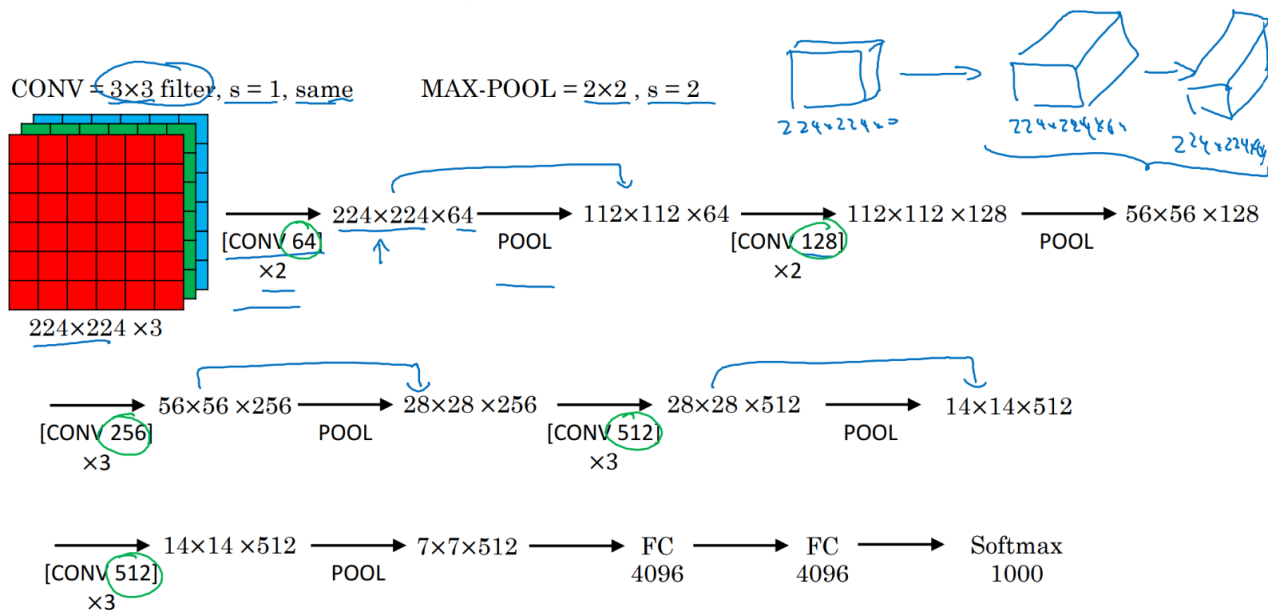
• AlexNet

- Three channel images $\rightarrow$ 96 11×11 filters with stride 4
- Similar to LeNet but much bigger (60 mil parameters), used ReLU
- Multiple GPUs (slower at the time), archaic local response normalization layer (at one position of height and width, normalize numbers across all channels)



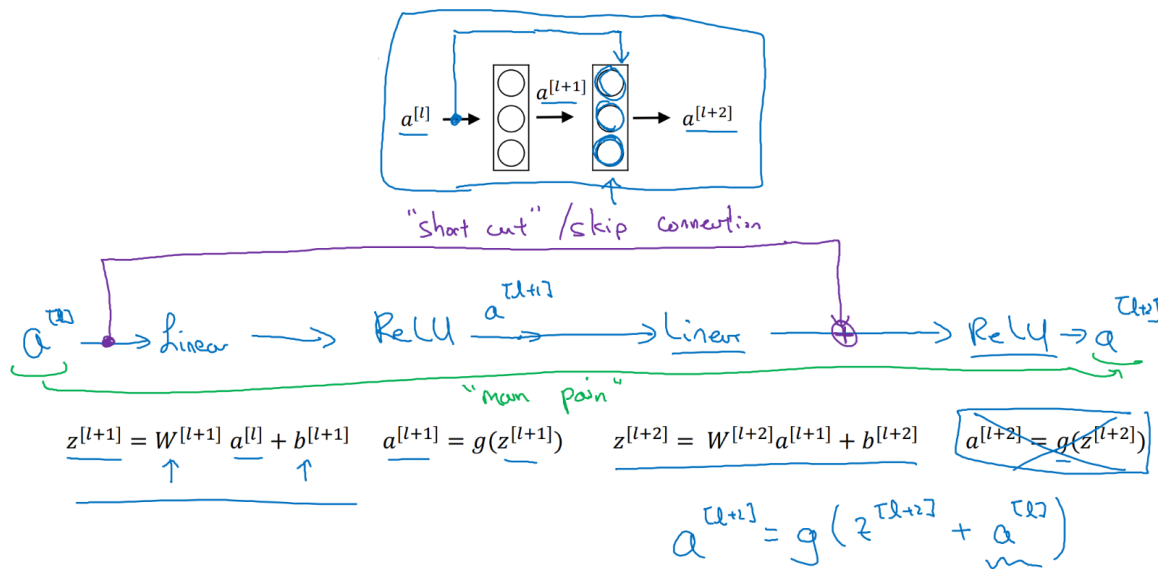
- **VGG (VGG-16)**

- Simpler architecture where you focus on having conv layers that are just 3x3 filters with stride 1 and same padding and 2x2 max pooling layers with stride 2
- 16 layers with weights; 138 mil parameters
- Height and width halves, number of filters doubles
- VGG-19 also exists

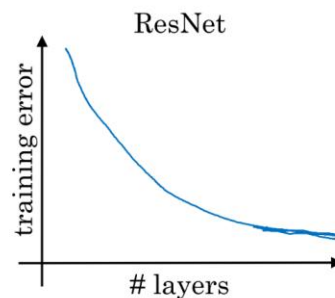
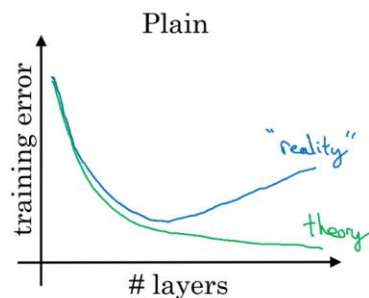
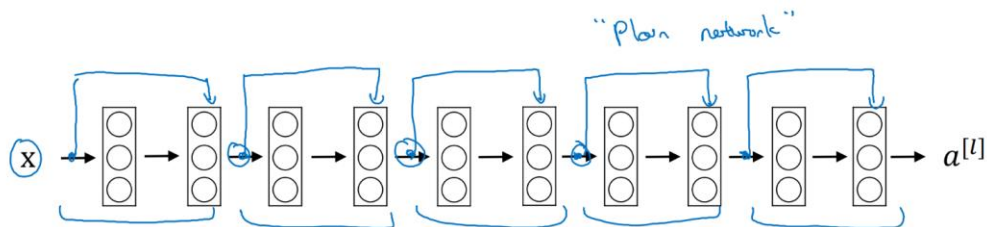


Residual Networks (ResNets)

- Very deep neural networks are difficult to train due to vanishing and exploding gradients
- **Skip connections** allow you to take activations from one layer and feed into another layer much deeper in the neural network
- **Residual block**
 - Main path: $a^{[l]}$ feeds into linear which feeds into ReLU, etc
 - Skip connection: Add $a^{[l]}$ before applying the next non-linearity (ReLU)
 - Using residual blocks allows you to train much deeper NNs

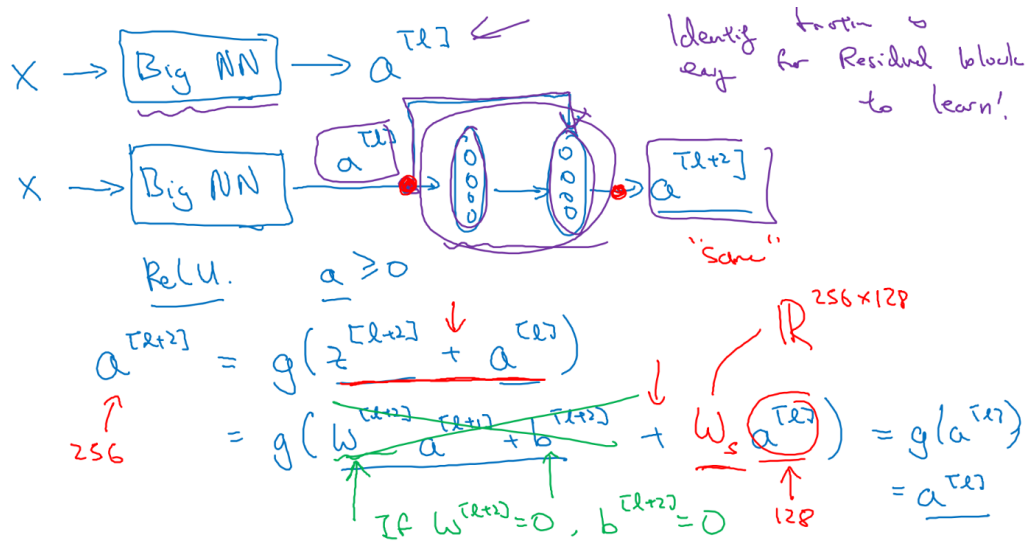


- Residual network
 - Stack several residual blocks together to form a deep network
 - Add skip connections to plain network

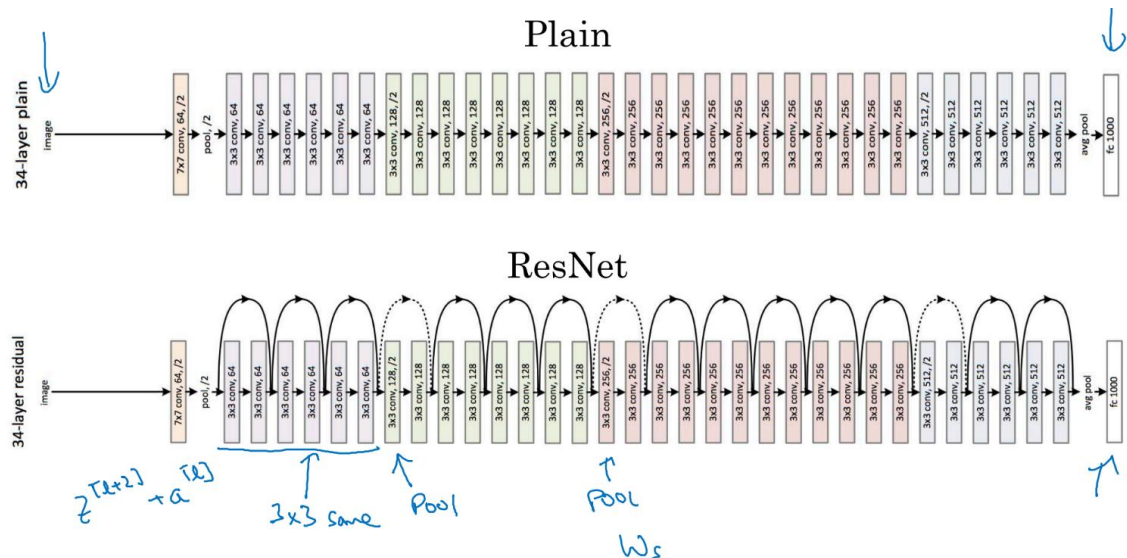


- Why ResNets work

- If using L2 regularization, that will tend to shrink the value of $W^{[l+2]}$. If = 0, you are left with only $g(a^{[l]}) = a^{[l]}$ (ReLU of non-negative value is the value)
- Identity function is easy for residual block to learn (hard for plain deep NNs)
- Adding two extra layers at middle/end of big NN doesn't hurt its performance. If they learn something useful, you can do even better than learning the identity fxn
- Assumes $z^{[l+2]}$ and $a^{[l]}$ are the same dimensions $\rightarrow$ use "same" convolutions or you can add an extra matrix W_s to fix the dimension of $a^{[l]}$

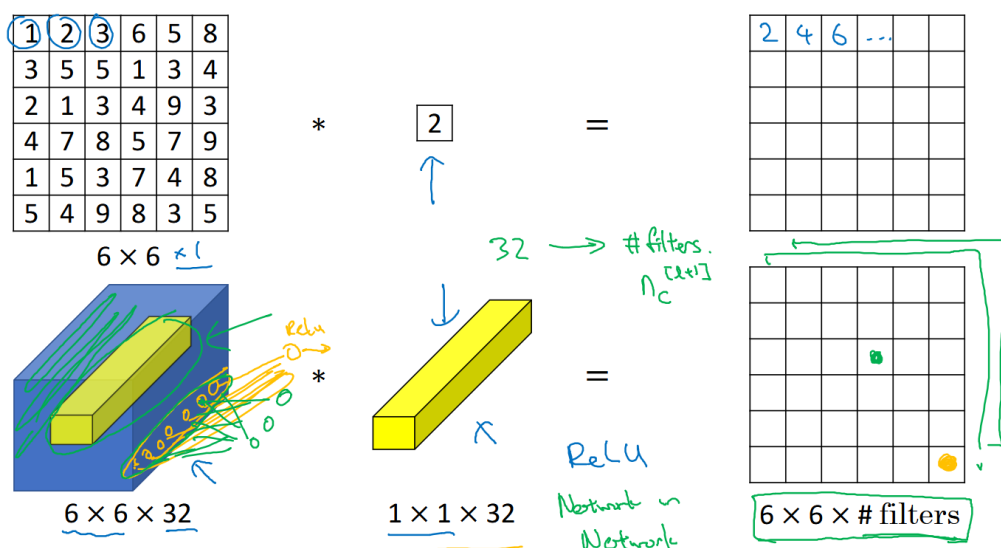


- 3x3 same convolutions preserve dimensions
- Conv conv conv pool ... FC $\rightarrow$ softmax

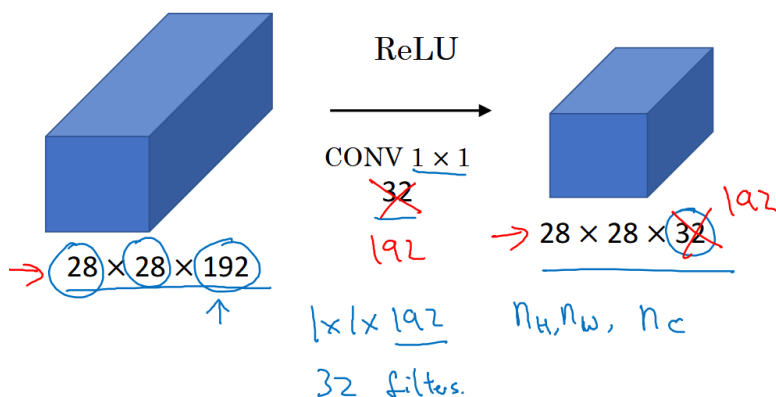


Network in Network and 1x1 Convolutions

- What does a 1×1 convolution do?
 - Looks at each of the $h \times w$ positions and take the element-wise product between the 32 numbers on the left and the 32 numbers in the filter, and then apply a ReLU
 - One slice through volume (one neuron that takes 32 #s as input, multiplies each of the 32 #s in one slice/in the same position by 32 weights, applying a ReLU non-linearity to it, and then outputting the corresponding result)
 - Multiple filters: multiple neurons taking the #s in one slice as inputs and building them up into an output
 - Fully connected neural network that applies to each of the 62 different positions and inputs 32 #s and outputs # filters $n_c^{[l+1]}$

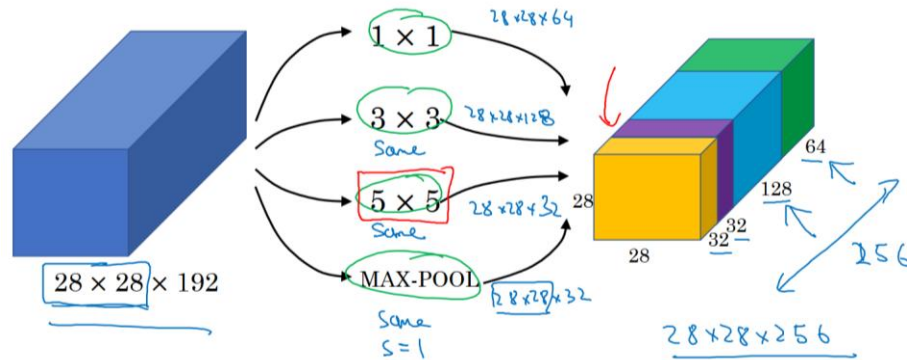


- Using 1×1 convolutions
 - Shrink # channels** by using 32 $1 \times 1 \times 192$ filters and output $28 \times 28 \times 32$
 - If not shrinking # channels, just applies non-linearity (complexity)

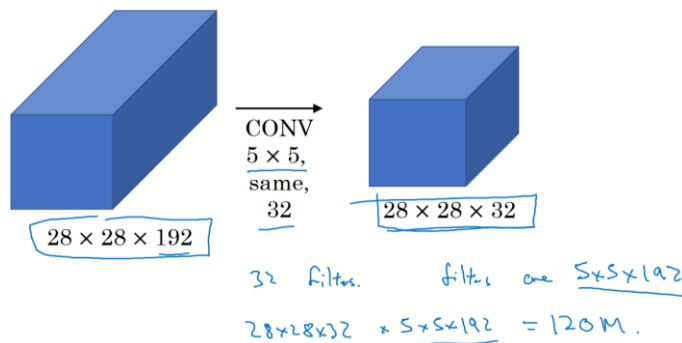


Inception Network

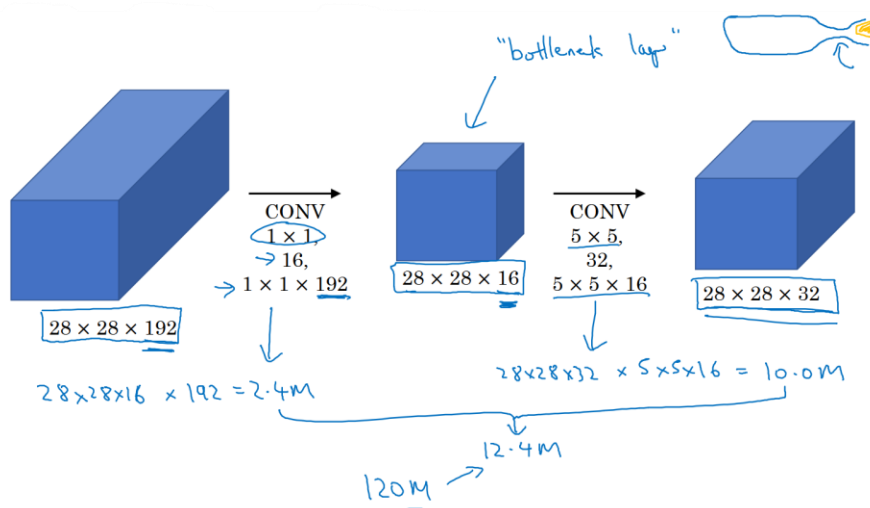
- Normally you have to pick what layer dimensions/type. Inception does them all and concatenates the results
- Let the network learn whatever parameters it wants to use (combinations of filter sizes)
- Pooling needs padding to concatenate with the others



- Computational cost

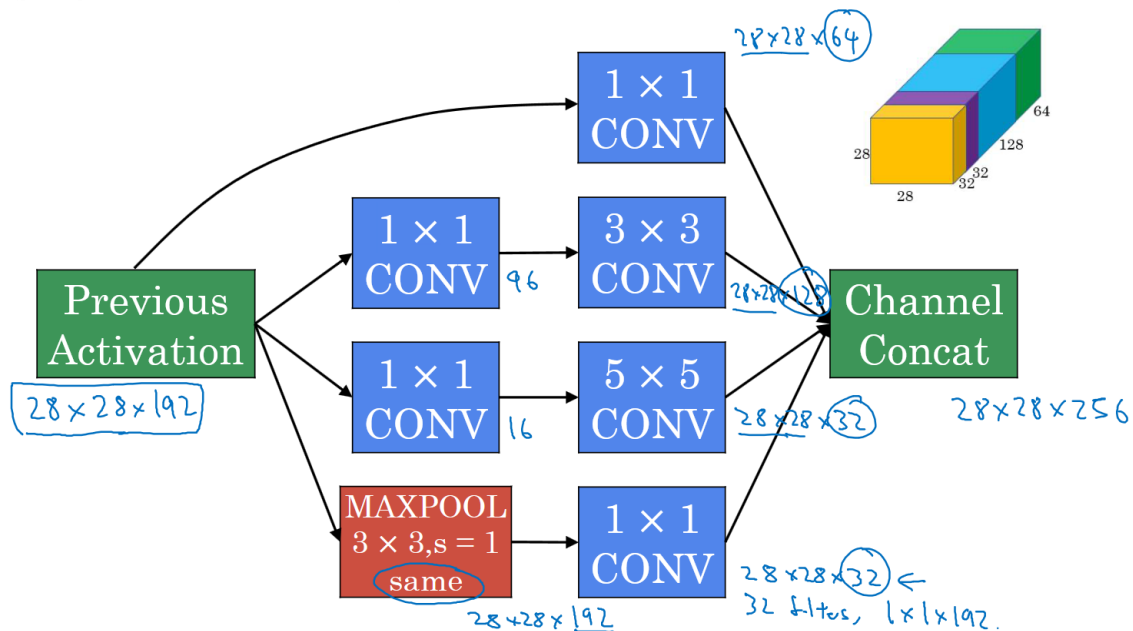


- Using 1×1 convolution
 - Much smaller intermediate volume (bottleneck layer), then apply bigger conv

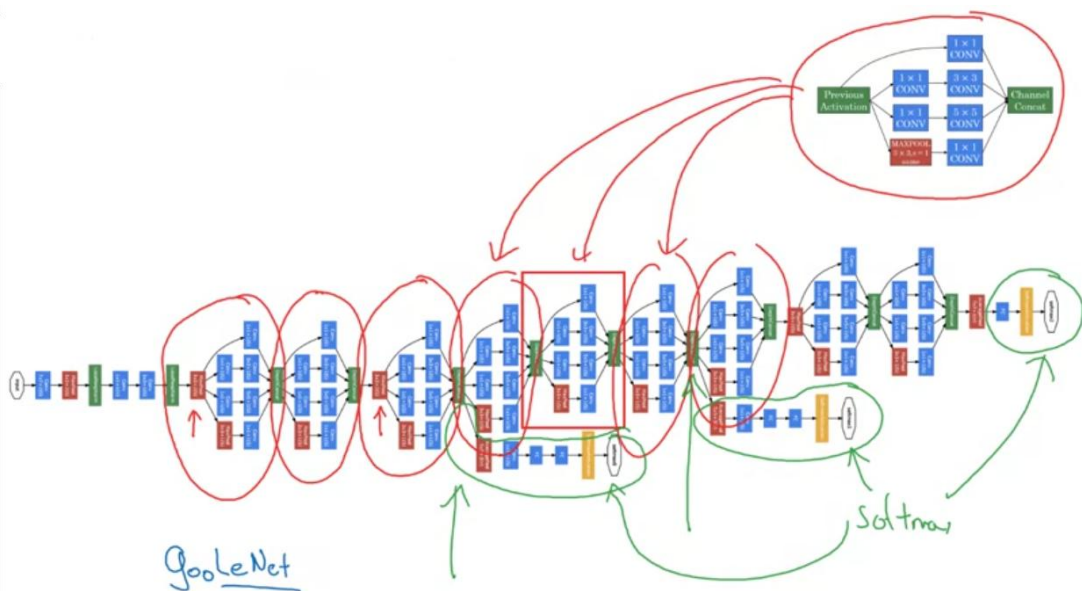


$1/10^{\text{th}}$ as many

- Inception module
 - Even with padding, pooling needs a 1×1 conv to shrink the # of channels



- Inception network
 - Puts inception modules together, many repeated blocks
 - Side branches use some hidden layer to try to make predictions. Helps ensure feature computed, even in hidden units in intermediate layers, aren't too bad for predicting the output. Has a regularizing effect on the inception network and helps prevent overfitting



MobileNet

- Low computational cost at deployment
- Useful for mobile and embedded vision applications
- Key idea: Normal vs. depthwise-separable convolutions
- Normal Convolution

Diagram illustrating Normal Convolution:

Input: $6 \times 6 \times 3$ ($n \times n \times n_c$)

Filter: $3 \times 3 \times 3$ ($f \times f \times n_c$) with n_c filters (5)

Output: $4 \times 4 \times 5$ ($n_{out} \times n_{out} \times n_c$)

Computational cost calculation:

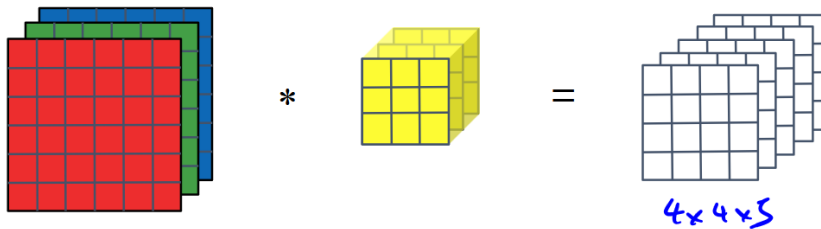
$$\text{Computational cost} = \frac{\# \text{filter params}}{3 \times 3 \times 3} \times \frac{\# \text{filter positions}}{4 \times 4} \times \# \text{of filters}$$

$$2160 = 3 \times 3 \times 3 \times 4 \times 4 \times 5$$

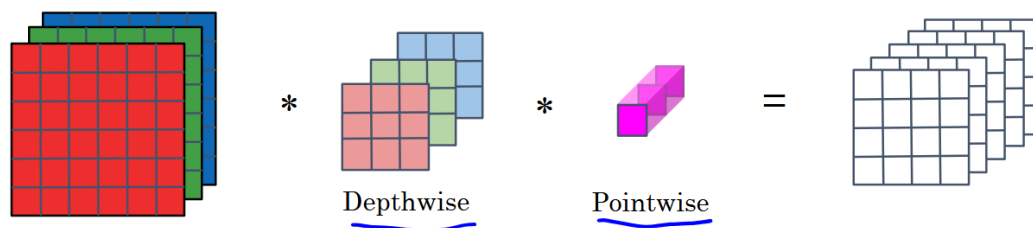
Depthwise Separable Convolution

- Two steps: depthwise convolution, then pointwise convolution

Normal Convolution

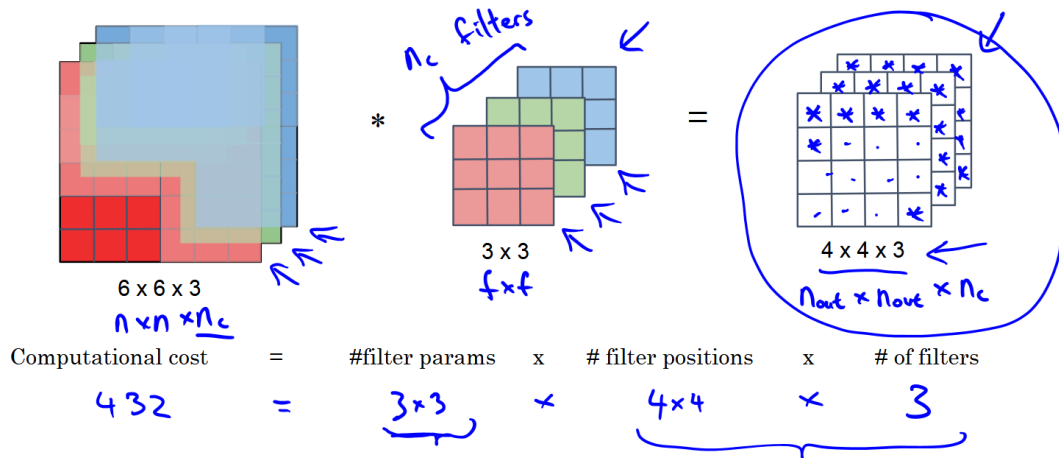


Depthwise Separable Convolution



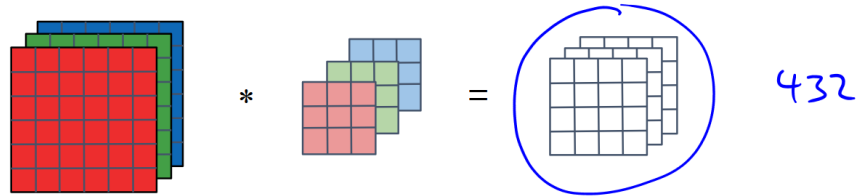
- **Step 1: Depthwise Convolution**

- Apply one of each of the filters to one of each of the input channels
- Red filter applies to 1st channel of input (9 multiplications), green to 2nd, etc

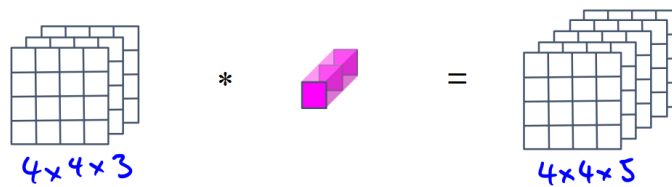


- Next, need to take this 4x4x3 intermediate value and carry out one more step to get the 4x4x5 output we want

Depthwise Convolution

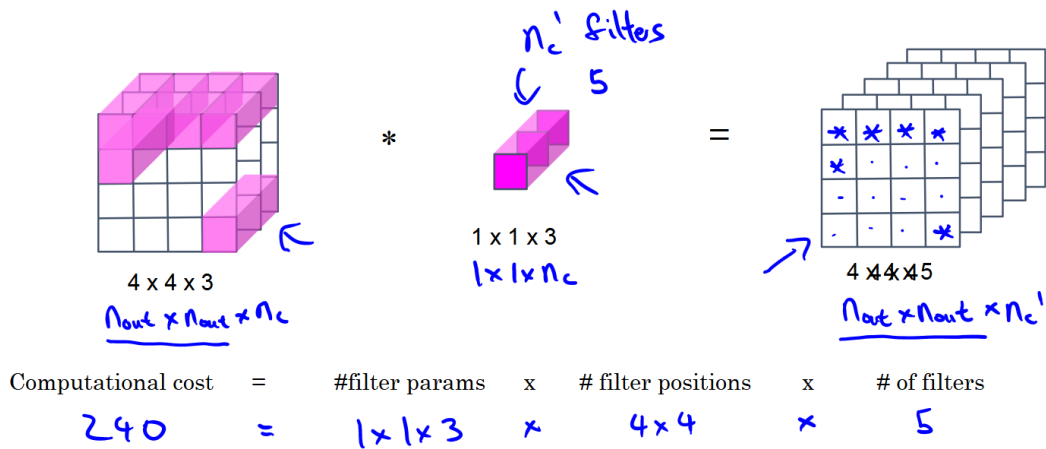


Pointwise Convolution



- Step 2: **Pointwise Convolution**

- Apply the $1 \times 1 \times 3$ filter to the same position in all channels of the input



- Cost Summary

- Depthwise separable was 31% as computationally expensive as normal conv
- Generally, DS is $1/n_c' + 1/f^2$ as computationally expensive as normal conv

Cost of normal convolution 2160

Cost of depthwise separable convolution

depthwise + pointwise
432 + 240 = 672

$\frac{672}{2160} = 0.31 \leftarrow$

$$= \frac{1}{n_c'} + \frac{1}{f^2}$$

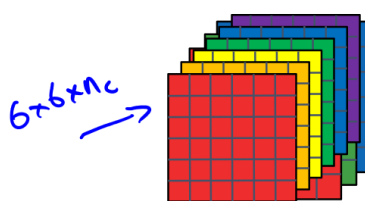
$$= \frac{1}{5} + \frac{1}{9}$$

$$= \frac{1}{512} + \frac{1}{3^2}$$

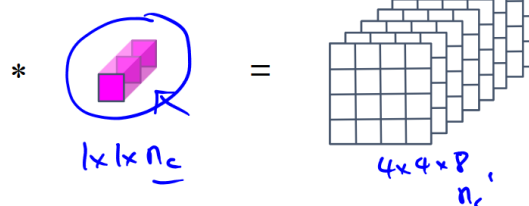
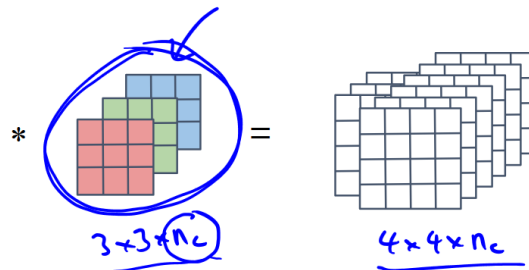
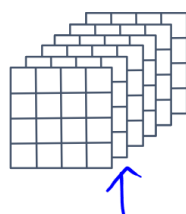
~ 10 times cheaper

- Depthwise separable convolution works for any number of input channels

Depthwise Convolution

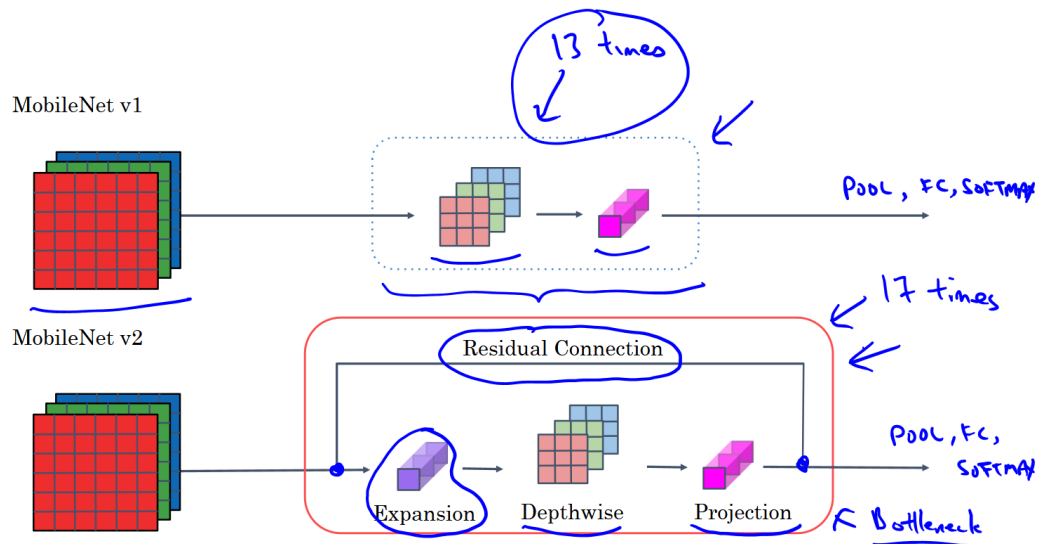


Pointwise Convolution

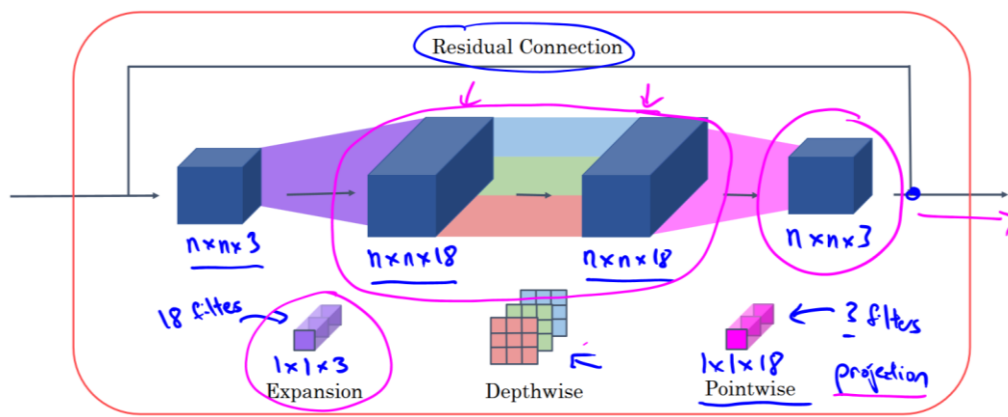


MobileNet Architecture

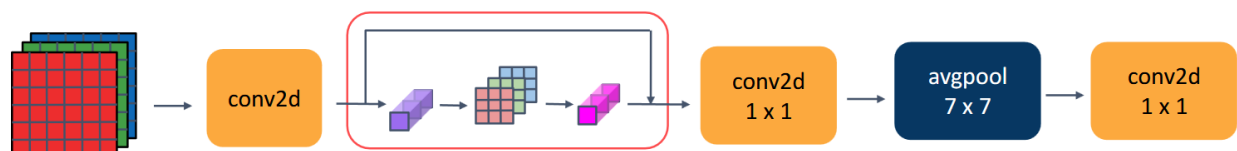
- MobileNet
 - Everywhere you would have used an expensive convolutional operation, instead use a less expensive depth wise separable convolution. Still performs well
 - V2 adds (a) residual/skip connections allowing for more efficient backprop and (b) expansion layer before DS



- MobileNet v2 bottleneck
 - Pass input directly to output and to expansion layer (commonly factor is 6)
 - Bottleneck block uses pointwise convolution (projection operation) to project back down to a smaller set of values so that amount of memory need to pass to the next block is reduced

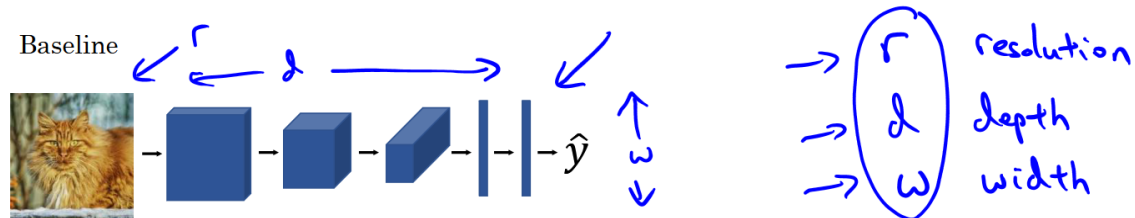


- MobileNet v2 Full Architecture

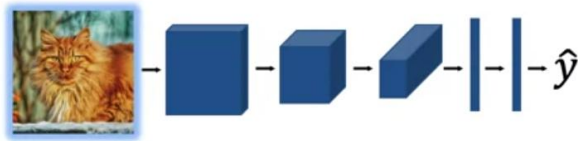


EfficientNet

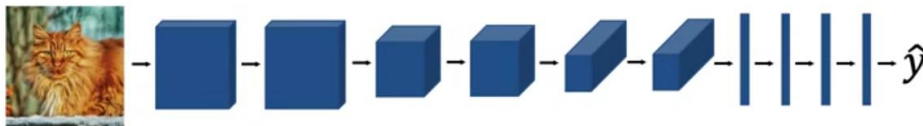
- Tune MobileNet to a specific device/automatically scale NNs up or down
- Given a computational budget, what are good choices of resolution, depth, and width?
- **Compound scaling:** simultaneously scale up or down the resolution of the image and depth/width of the neural network



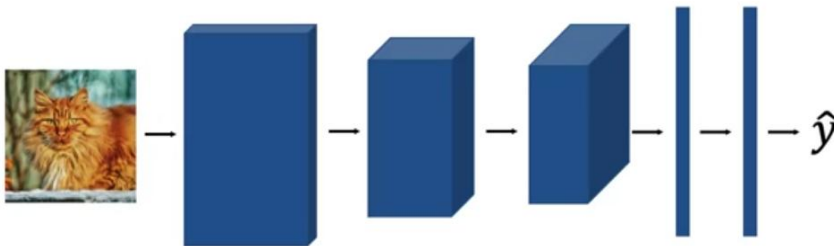
Higher Resolution



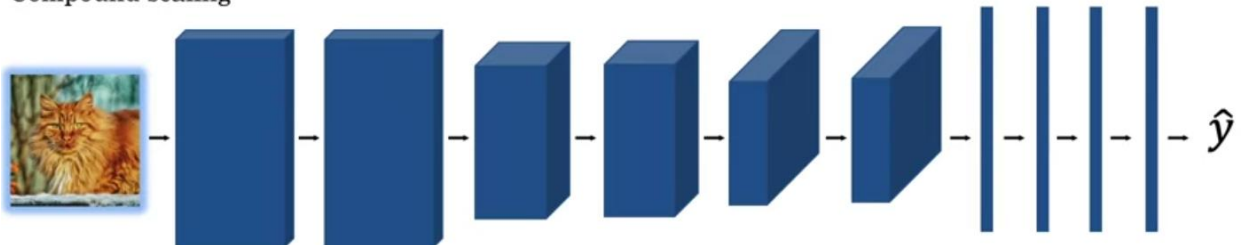
Deeper



Wider



Compound scaling

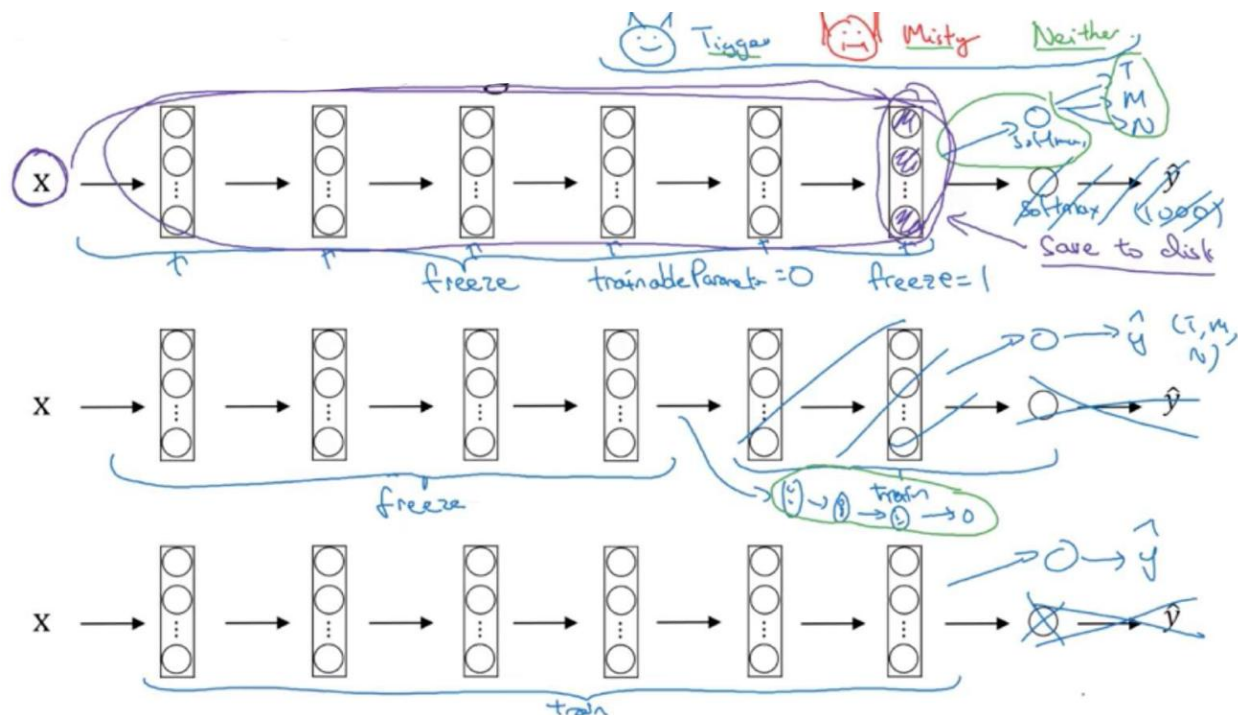


Using Open-Source Implementation

- Faster to use author's implementation and going from there than to reimplement from scratch
- Check and use GitHub. In command shell: `git clone [URL]`

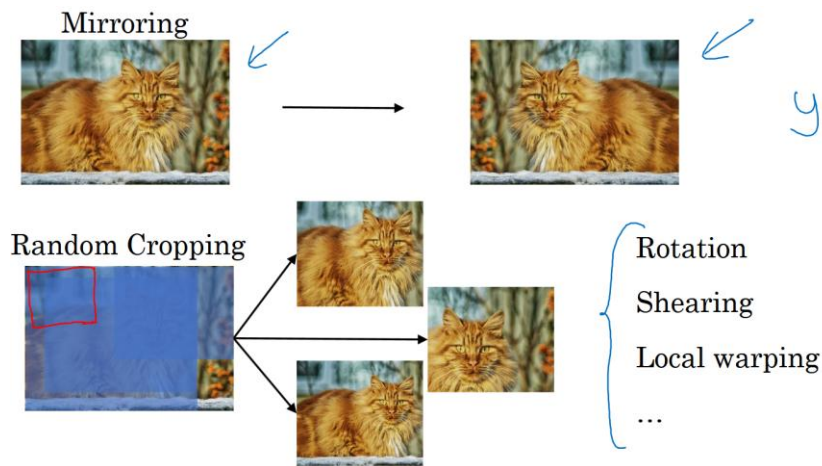
Transfer Learning

- No need to initialize from scratch; use online resources as pre-training to get going
- Download code/weights from a similar classifier. Replace softmax layer with your own
- Freeze parameters in downloaded network and just train the parameters associated with the softmax layer
- Other trick: there exists a function (fixed because you're not training it) that maps the input x to some set of activation in the last layer. Pre-compute that layer's activations and save them to disk. Training shallow softmax model from feature vector to make a prediction
- Larger training set: freeze fewer layers and train later layers or make new
- Huge training set: take open-source network/weights, use the whole thing just as initialization, and train the whole network

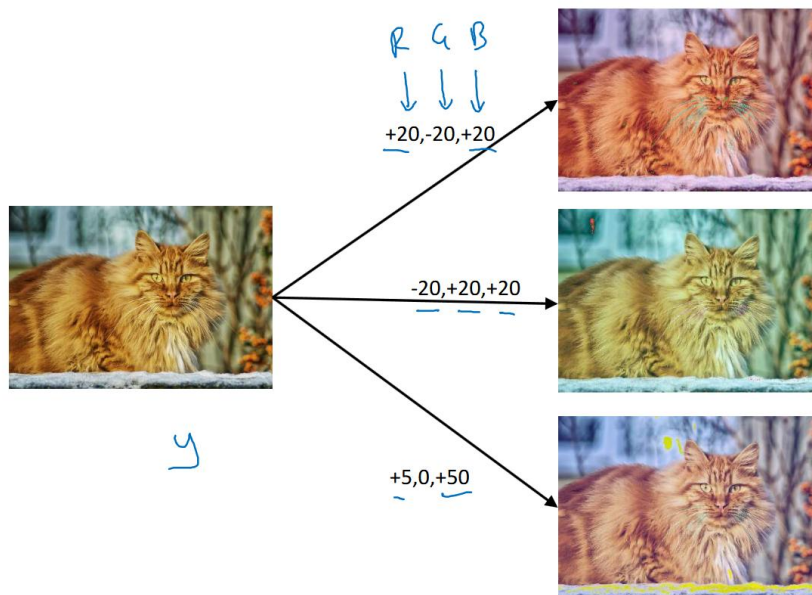


Data Augmentation

- More data to improve system performance (can't get enough data for computer vision)
- Common methods: mirroring, random cropping, rotation, shearing, local warping, etc.
- Note on random cropping: not truly random, some subsets are not useful. Just looking for reasonably large subsets of the actual image



- Color shifting
 - Add distortions to R G and B channels
 - In practice you draw RGB from some probability distribution (could be smaller)
 - Makes algorithm more robust to changes in colors of images
 - Different way to sample RGB: PCA color augmentation



Advanced:

PCA

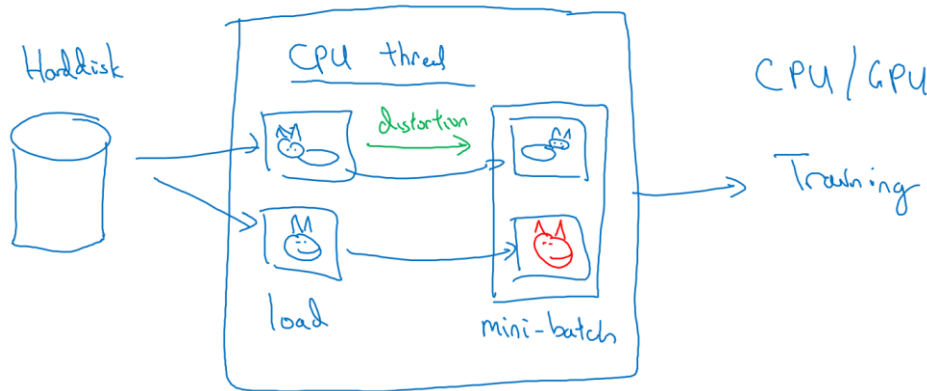
m2-class.org

[AlexNet paper

"PCA color augmentation."

R B G

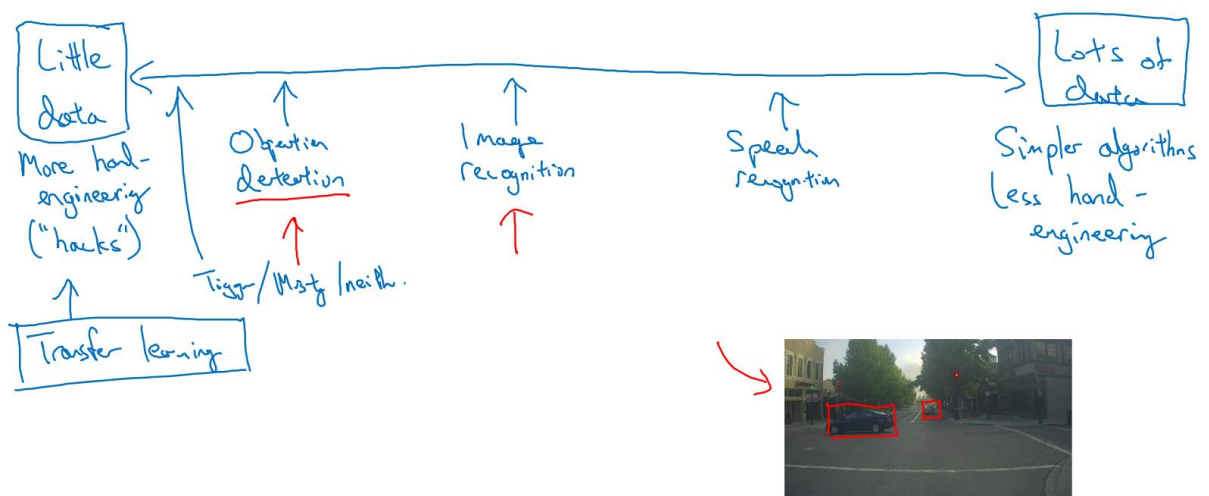
- Implementing distortions during training
 - CPU thread constantly loading images off hard disk
 - Use a CPU thread to implement distortions (form a mini-batch of data)
 - Data constantly pass to some other thread/process for implementing training
 - Separate threads for data and for training, can often run in parallel



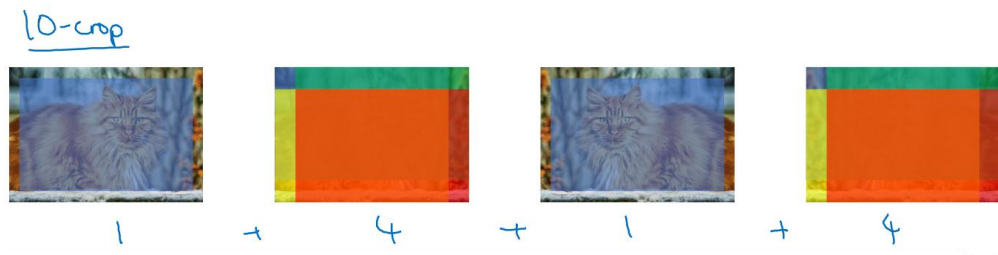
- Hyperparameters: how much color shifting, params for random cropping

The State of Computer Vision

- Two sources of knowledge:
 - Labeled data (x, y)
 - Lots of data → simpler algorithms and less hand-engineering
 - Hand engineering features/network architecture/other components
 - Little data → more hand-engineering, complex algorithms, transfer learning



- Tips for doing well on benchmarks/winning competitions
 - **Ensembling:** Train several networks (3-15) independently and average their outputs ($\hat{y}$). Requires lots of computer memory
 - **Multi-crop** at test time: run classifier on multiple versions of test images and average results. Not as much run memory, but slows run time
 - 10-crop method: crop center and 4 corners of image and its mirror



- Use open source code
 - Use architectures of networks published in the literature
 - Use open source implementations if possible
 - Use pretrained models and fine-tune on your dataset

When building a ConvNet, typically you start with some POOL layers followed by some CONV layers. True/False?

False True

Correct. It is typical for ConvNets to use a POOL layer after some Conv layers; sometimes even one POOL layer after each CONV layer; but is not common to start with POOL layers.

Which of the following do you typically see in a ConvNet? (Choose all that apply.)

FC layers in the last few layers

Multiple CONV layers followed by a POOL layer

Multiple POOL layers followed by a CONV layer

FC layers in the first few layers

LeNet - 5 made extensive use of padding to create valid convolutions, to avoid increasing the number of channels after every convolutional layer. True/False?

True **False**

The motivation of Residual Networks is that very deep networks are so good at fitting complex functions that when training them we almost always overfit the training data. True/False?

False True

Correct, very deep neural networks are hard to train and a deeper network does not always imply lower training error. Residual Networks allow us to train very deep neural networks.

The computation of a ResNet block is expressed in the equation:

$$a^{[l+2]} = g \left(\underbrace{W^{[l+2]}}_{\text{C}} g \left(\underbrace{W^{[l+1]} a^{[l]} + b^{[l+1]}}_{\text{A}} \right) + \underbrace{b^{[l+2]} + a^{[l]}}_{\text{B}} \right)$$

Which part corresponds to the skip connection?

The term in the red box, marked as C.

The equation of ResNet.

The term in the orange box, marked as B.

The term in the blue box, marked as A.

Which of the following equations captures the computations in a ResNet block?

$$a^{[l+2]} = g(W^{[l+2]} g(W^{[l+1]} a^{[l]} + b^{[l+1]}) + b^{[l+2]} + a^{[l]}) + a^{[l+1]}$$

$$a^{[l+2]} = g(W^{[l+2]} g(W^{[l+1]} a^{[l]} + b^{[l+1]}) + b^{[l+2]} + a^{[l]})$$

$$a^{[l+2]} = g(W^{[l+2]} g(W^{[l+1]} a^{[l]} + b^{[l+1]}) + b^{[l+2]}) + a^{[l]}$$

$$a^{[l+2]} = g(W^{[l+2]} g(W^{[l+1]} a^{[l]} + b^{[l+1]}) + b^{[l+2]})$$

Adding a ResNet block to the end of a network makes it deeper. Which of the following is true?

The performance of the networks is hurt since we make the network harder to train.

The number of parameters will decrease due to the shortcut connections.

It shifts the behavior of the network to be more like the identity function.

The performance of the networks doesn't get hurt since the ResNet block can easily approximate the identity function.

In a ResNet block the computations are given by $a^{[l+2]} = g(W^{[l+2]} a^{[l+1]} + b^{[l+2]} + a^{[l]})$ thus if $W^{[l+2]}$ and $b^{[l+2]}$ are zero then we get the identity function.

Which ones of the following statements on Residual Networks are true? (Choose all that apply.)

Using a skip-connection helps the gradient to backpropagate and thus helps you to train deeper networks

The skip-connection makes it easy for the network to learn an identity mapping between the input and the output within the ResNet block.

A ResNet with L layers would have on the order of L^2 skip connections in total.

The skip-connections compute a complex non-linear function of the input to pass to a deeper layer in the network.

Suppose you have an input volume of dimension $n_H \times n_W \times n_C$. Which of the following statements do you agree with? (Assume that the “1x1 convolutional layer” below always uses a stride of 1 and no padding.)

You can use a 2D pooling layer to reduce n_H , n_W , and n_C .

You can use a 1x1 convolutional layer to reduce n_C but not n_H and n_W .

You can use a 1x1 convolutional layer to reduce n_H , n_W , and n_C .

You can use a 2D pooling layer to reduce n_H , n_W , but not n_C .

For a volume of $125 \times 125 \times 64$ which of the following can be used to reduce this to a $125 \times 125 \times 32$ volume?

Use a POOL layer of size 2×2 with a stride of 2.

Use a POOL layer of size 2×2 but with a stride of 1.

Use a 1×1 convolutional layer with a stride of 2, and 32 filters.

Use a 1×1 convolutional layer with a stride of 1, and 32 filters.

Using 1×1 convolutions is a great way to reduce the depth dimension without affecting the other dimensions.

Which ones of the following statements on Inception Networks are true? (Choose all that apply.)

Inception networks incorporate a variety of network architectures (similar to dropout, which randomly chooses a network architecture on each step) and thus has a similar regularizing effect as dropout.

Making an inception network deeper (by stacking more inception blocks together) can improve performance, but can also lead to overfitting and increase in computational cost.

A single inception block allows the network to use a combination of 1x1, 3x3, 5x5 convolutions and pooling.

Inception blocks usually use 1x1 convolutions to reduce the input data volume's size before applying 3x3 and 5x5 convolutions.

Which of the following are true about bottleneck layers? (Choose all that apply)

By adding these layers we can reduce the computational cost in the inception modules.

Using the 1×1 convolutional layers we can reduce the depth of the volume and help reduce the computational cost of applying other convolutional layers with different filter sizes

Bottleneck layers help to compress the 1x1, 3x3, 5x5 convolutional layers in the inception network.

The bottleneck layer has a more powerful regularization effect than Dropout layers.

The use of bottlenecks doesn't seem to hurt the performance of the network.

significantly reduces the computational cost

Models trained for one computer vision task can't be used directly in another task. In most cases, we must change the softmax layer, or the last layers of the model and re-train for the new task. True/False?

False

True

When having a small training set to construct a classification model, which of the following is a strategy of transfer learning that you would use to build the model?

Use an open-source network trained in a larger dataset freezing the layers and re-train the softmax layer.

It is always better to train a network from a random initialization to prevent bias in our model.

Use an open-source network trained in a larger dataset, freeze the softmax layer, and re-train the rest of the layers.

Use an open-source network trained in a larger dataset. Use these weights as an initial point for the training of the whole network.

In Depthwise Separable Convolution you:

Perform two steps of convolution.

The final output is of the dimension $n_{out} \times n_{out} \times n_c$ (where n_c is the number of filters used in the pointwise convolution step).

For the “Depthwise” computations each filter convolves with only one corresponding color channel of the input image.

Perform one step of convolution.

The final output is of the dimension $n_{out} \times n_{out} \times n_c$ (where n_c is the number of color channels of the input image).

For the “Depthwise” computations each filter convolves with all of the color channels of the input image.

You convolve the input image with n_c number of $n_f \times n_f$ filters (n_c is the number of color channels of the input image).

You convolve the input image with a filter of $n_f \times n_f \times n_c$ where n_c acts as the depth of the filter (n_c is the number of color channels of the input image).

Which of the following are true about Depthwise-separable convolutions? (Choose all that apply)

The depthwise convolution convolves each channel in the input volume with a separate filter. Output of this kind of convolution is the same as the input

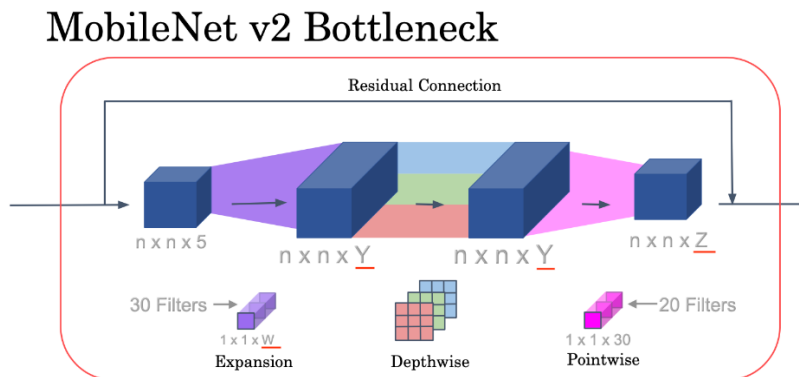
Depthwise-separable convolutions are composed of two different types of convolutions. Depthwise → Pointwise

The pointwise convolution convolves the output volume with 1×1 filters.

The number of filters for the output of the depthwise-separable convolution is determined by the number of 1×1 filters used.

The depthwise convolution convolves the input volume with 1×1 filters over the depth dimension.

Fill in the missing dimensions shown in the image below (marked W, Y, Z).



W = 5, Y = 20, Z = 5

W = 30, Y = 20, Z = 20

W = 30, Y = 30, Z = 5

W = 5, Y = 30, Z = 20

Suppose that in a MobileNet v2 Bottleneck block the input volume has shape $64 \times 64 \times 16$. If we use 32 filters for the expansion and 16 filters for the projection. What is the size of the input and output volume of the depthwise convolution, assuming a pad='same'?

$64 \times 64 \times 16, 64 \times 64 \times 32$

$32 \times 32 \times 32, 32 \times 32 \times 32$

$64 \times 64 \times 32, 64 \times 64 \times 32$

$64 \times 64 \times 32, 64 \times 64 \times 16$

$64 \times 64 \times 16 \rightarrow (1 \times 1 \text{ expansion, 32 filters}) \rightarrow 64 \times 64 \times 32 \rightarrow (\text{depthwise conv, same padding}) \rightarrow 64 \times 64 \times 32 \rightarrow (1 \times 1 \text{ projection, 16 filters}) \rightarrow 64 \times 64 \times 16$

The size of the input and output volume of the depthwise convolution is determined by the number of filters in the expansion.